UNIT E — Undecidable Problems (Why Perfect Tools Do Not Exist) . . . 3
Study Notes (Book Chapter) . . . 3
How to use these notes . . . 3
Before you begin — the big picture . . . 4
1. The undecidable family — problems about programs (E.1.1) . . . 6
1.1 Scenario: the requirements meeting that asked for three impossible things . . . 6
1.2 Simple analogy: the magic tasting contest . . . 6
1.3 From first principles: three kinds of questions, and where the impossible ones live 7
1.4 The big three: program equivalence, behavioural identity, and specification satisfaction . . . 8
1.4.1 Program equivalence — “do these two programs behave identically?” . . . 8
1.4.2 Specification satisfaction — “does this program meet its specification?” . . . 9
1.4.3 Behavioural identity — “do these two systems produce the same output for every input?” . . . 9
1.5 The family portrait — and the family rule . . . 9
1.6 Why the whole family is undecidable — the reason in one sentence . . . 11
1.7 Where you will use this (E.1.1) . . . 11
1.8 Self-check (E.1.1) . . . 12
2. Classifying problems: decidable or undecidable — the reduction technique (E.1.2) . . . 12
2.1 Scenario: the triage meeting — ten feature requests, one hour . . . 12
2.2 Simple analogy: the two baskets and the referee . . . 13
2.3 The classification protocol — the procedure, step by step . . . 14
2.4 Worked classifications — the gallery . . . 16
Worked 1 — “Validate this phone number” — DECIDABLE . . . 16
Worked 2 — “Are these two configuration files semantically identical?” — DECIDABLE . . . 16
Worked 3 — “Does this file contain a `TODO` comment?” — DECIDABLE . . . 16
Worked 4 — “Does this program behave identically to that one?” (program equivalence) — UNDECIDABLE . . . 17
Worked 5 — “Does this program ever print `ERROR`?” — UNDECIDABLE . . . 17
Worked 6 — “Does this program ever read from the network?” — UNDECIDABLE . . . 17
Worked 7 — “Does this program halt on an empty input?” — UNDECIDABLE . . . 18
Worked 8 — “Does this program ever terminate?” (the totality problem) — UNDECIDABLE . . . 18
Worked 9 — “Does this SQL query finish within 2 seconds?” — DECIDABLE (with a nuance) . . . 18
Worked 10 — “Is this program a virus?” — UNDECIDABLE . . . 19
2.5 The decision table — the gallery at a glance . . . 19
2.6 The honest shortcuts — recognising the shape without building a wrapper . . . 21
2.7 Where you will use this (E.1.2) . . . 22
2.8 Self-check (E.1.2) . . . 22

3. Rice's theorem — one proof for every behavioural property (E.1.3) 23
3.1 Scenario: the CEO who wants the one tool that answers everything 23
3.2 Simple analogy: the snail messenger 23
3.3 From first principles: what "a property of a program's behaviour" means 24
3.4 Rice's theorem — stated 25
3.5 The proof — the snail in formal dress 26
3.6 Applying Rice's theorem — the instant results 27
3.7 What Rice's theorem does NOT say — the boundaries 29
3.8 Where you will use this (E.1.3) 30
3.9 Self-check (E.1.3) 31
4. The practical implications — why perfect tools do not exist, and what industry does instead (E.1.4) 31
4.1 Scenario: the impossible promises meeting 31
4.2 Simple analogy: the bridge inspector 32
4.3 The three impossible products — and their exact theorem 33
4.4 The evidence-based toolkit — what industry actually ships 33
4.5 The trust ladder — what you can honestly promise 35
4.6 The professional synthesis — evidence engineering 36
4.7 Where you will use this (E.1.4) 37
4.8 Self-check (E.1.4) 38
5. Chapter summary and criteria checklist 38
5.1 The chapter in ten takeaways 38
5.2 Criteria checklist (use this to self-test before any assessment) 39
6. Glossary of key terms 40
7. Self-assessment questions (answers in Section 7.6) 43
7.1 On E.1.1 43
7.2 On E.1.2 43
7.3 On E.1.3 43
7.4 On E.1.4 44
7.5 On all criteria (integrated) 44
7.6 Answers to self-assessment 44
8. Exam-style question bank 47
8.1 Multiple choice 47
8.2 Short questions (2–4 marks each) 48
8.3 Medium questions (5–8 marks each) 49
8.4 Long questions (10–15 marks each) 49
8.5 Applied/scenario question 50
8.6 Mark allocation suggestion 50
8.7 Model answers (outline form) 51
9. Sources and further reading 54

UNIT E — Undecidable Problems (Why Perfect Tools Do Not Exist)

Study Notes (Book Chapter)

Module: TCR117V — Theoretical Computer Science V (Advanced Diploma, FoICT, TUT) **Outcome covered:** E.1 — Identifying problems that no algorithm can solve (why perfect tools do not exist) **Assessment criteria:** E.1.1 – E.1.4 **Semester:** 2, 2026

How to use these notes

This chapter is written as a textbook chapter. Each of the four assessment criteria (E.1.1 to E.1.4) forms its own major section, so you can study criterion by criterion:

Section	Assessment criterion	Topic
1	E.1.1	The undecidable family — problems about programs (program equivalence, behavioural identity, specification satisfaction)
2	E.1.2	Classifying problems as decidable or undecidable — the reduction technique, with worked classifications
3	E.1.3	Rice’s theorem — one proof that every non-trivial behavioural property is undecidable
4	E.1.4	The practical implications — why perfect

Section	Assessment criterion	Topic
		virus detection, complete bug detection and fully automated verification are impossible, and what industry does instead
5	—	Chapter summary and criteria checklist
6	—	Glossary of key terms
7	—	Self-assessment questions (answers at the end)
8	—	Exam-style question bank (model answers at the end)
9	—	Sources and further reading

Every section follows the same pattern: a **real-world scenario** first, then a **simple analogy** you can tell a child, then the **concepts** built up from first principles, then **worked examples**, then a **conclusion-first summary**, and finally a **“Where you will use this”** box that links the theory to the working world. Key structures are drawn as figures (the family of undecidable problems, the classification protocol, the reduction recipe, the worked classification gallery, Rice’s theorem and its proof, the boundaries of the theorem, the evidence-based toolkit, and the map of the unit). The mathematics in this unit — functions and partial functions, properties of behaviour, logical equivalence in reductions, and the shape of contradiction proofs — is explained step by step from the very beginning: nothing is assumed, and nothing is skipped. Work through the **self-check questions** as you go — they are designed to show you exactly what you would be asked in a test or exam.

Before you begin — the big picture

In Unit D you met the Halting Problem: no program can decide, for every program and every input, whether the program ever stops. That was one famous result — one specific

question that no algorithm can answer. This unit does something far more unsettling: it shows that the Halting Problem is not a lonely exception but the *typical case*. **Almost every interesting question you can ask about what a program does has no algorithm at all.**

Think about the questions real software people ask every day:

- “Do these two programs behave identically?” (The question behind safe refactoring: the new version must do exactly what the old one did.)
- “Does this program meet its specification?” (The question behind verification, audits and certification.)
- “Is this file a virus?” (The question behind every antivirus product.)
- “Is this code free of bugs?” (The question behind QA, testing and code review.)
- “Does this program always terminate?” (The question behind critical systems, from medical devices to aircraft control.)

Each of these, asked in its perfect “answer for every program” form, is **undecidable**: no algorithm can answer it for all programs, ever — not with more cleverness, not with faster hardware, not with AI. This unit explains *why* (Sections 1–3) and, just as importantly, what professionals actually do about it (Section 4) — because the industry does not collapse: it ships antivirus products, verification tools, and quality gates every day. The trick is that every honest tool answers a *smaller, decidable question* — and knowing the difference between the decidable question and the undecidable one is the difference between an honest product and a fraudulent guarantee.

Unit D gave you the two tools you will reuse constantly here: **diagonalisation** (the proof that one problem, HALT, is undecidable) and **reduction** (the machine that moves impossibility from one problem to another). This unit’s first half is about recognising the shape: *questions about the behaviour of programs are where undecidability lives* (E.1.1). The second half gives you the two professional instruments: the **classification protocol** that tells you, for any given problem, which side of the line it falls on (E.1.2), and **Rice’s theorem** — the master key that proves a whole family of undecidable problems in one stroke (E.1.3). The unit closes with the payoff: what all of this means for your career (E. 1.4). When a client asks “can you guarantee no bugs?”, or a vendor promises “100% malware detection”, you will be the person in the room who knows whether the promise is physics or marketing — and what to build instead.

By the end of this chapter you should be able to look at any problem statement and classify it: **decidable** (an algorithm that always terminates exists), **undecidable** (no algorithm can exist — by reduction or by Rice’s theorem), or **evidence-based** (no algorithm can exist, so the professional ships testing, analysis and heuristics with an honest guarantee). That three-way split is the most employment-relevant idea in this module — it decides which features can be built at all, and which must be built differently.

---

1. The undecidable family — problems about programs (E.1.1)

> **Assessment criterion E.1.1:** Examples of undecidable problems that arise in computing (e.g. program equivalence, whether two programs behave identically, whether a program satisfies a specification) are identified.

1.1 Scenario: the requirements meeting that asked for three impossible things

It is Monday morning, and the requirements meeting has three items on the board:

1. **The refactoring guarantee.** The company's payments module — 90 000 lines of battle-tested code — must be rewritten in a new framework. The board wants a tool that reads the old module and the new module and answers: *do these two programs behave identically?* If yes, the rewrite is safe to ship. Nobody wants to be the one who signs off without it.

2. **The compliance check.** A regulator demands that the system "satisfy its specification" — the formal document describing what the system must and must not do. The board wants a tool that reads any program and any specification and answers: *does this program meet this specification?*

3. **The behavioural identity audit.** Two suppliers bid for the same service: a scoring algorithm and its "improved" competitor. The board wants a tool that reads both and answers: *do they produce the same output for every possible input?* The business wants to switch suppliers without changing customer-facing behaviour.

These are not hypothetical or exotic requests. They are the natural questions of a mature software industry: safe refactoring, regulatory compliance, and drop-in replacement. Every one of them is the same kind of question — a question *about programs* — and every one of them, asked in its perfect "answer for all programs" form, has no algorithm. No tool can ever be built that answers any of them for every program. That is not a difficulty; it is a theorem — and the rest of this section explains why, starting from a story you could tell a child.

1.2 Simple analogy: the magic tasting contest

Tell this to a six-year-old:

Two chefs, Chef A and Chef B, each run a magic kitchen. A magic kitchen is strange: if you give it an ingredient, it cooks something — and there is no limit to how many dishes a kitchen can make. A judge is hired to answer one question: *do the two kitchens cook the same food?* That sounds fair. But here is the catch: "the same food" means the same dish

for **every** ingredient that could ever be brought in — millions and millions of ingredients, and the kitchens can cook forever.

The judge starts tasting: wheat flour and cinnamon in kitchen A gives a spiced cake; in kitchen B, the same. So far so good. But to *prove* the kitchens are identical, the judge must taste every possible dish — and there is no last ingredient. The tasting would never end. And a trickster chef knows this: kitchen B can be built so that for one special ingredient, it behaves oddly — but only after a very long, secret preparation. Even watching the kitchens forever could not settle the question, because the answer can depend on a secret event that never happens.

In grown-up terms: the two kitchens are two *programs*; each ingredient is an *input*; "the dish cooked" is the *output*. "Do these two programs behave identically?" means: for **every** input, do they halt with the same output (or both fail to halt)? Because there are infinitely many inputs, checking them one by one can never finish — and because a program's behaviour can be hidden behind another program's halting (the trickster chef), no amount of watching ever settles it. The question is not "hard to check" — it is *impossible to check in general*, and no algorithm exists. That is the shape of the whole undecidable family: **questions about what programs do, asked for all programs, are questions no program can answer.**

1.3 From first principles: three kinds of questions, and where the impossible ones live

Before the examples, the vocabulary. Every question you can ask in computing falls into one of three kinds, and they have very different fates. The distinction is the single most useful classification habit in this unit — learn it once and most classification questions (Section 2) answer themselves.

Kind 1 — Questions about data. "Does this string match this pattern?" "Is this number prime?" "Does this graph contain a cycle?" The input is *ordinary data*: strings, numbers, graphs. The question is answered by an algorithm that processes the data and terminates. These are the comfortable problems: they are **decidable** — an algorithm exists that always terminates with the correct answer. (Unit C's complexity theory is about how *fast* those algorithms are; this unit's question is whether they exist at all.)

Kind 2 — Questions about the text of a program. "Does this program have fewer than 100 lines?" "Does this program contain the word `TODO`?" "Does this program use the `goto` statement?" The input is a *program*, but the question is about its **text** — its syntax, its shape — not about what it does when it runs. Reading text and counting patterns is ordinary data processing: these questions are **decidable**. This is exactly what linters, style checkers, signature scanners and compilers' syntactic analysis do for a living.

Kind 3 — Questions about the behaviour of a program. "Does this program ever print `ERROR`?" "Does this program compute the same function as that one?" "Does this program ever terminate?" "Does this program satisfy its specification?" "Is this program a virus?" The input is a program, but the question is about **what the program does when it runs** — its behaviour. This is where undecidability lives. Every question of this kind,

asked in its general form (“answer for all programs”), is undecidable. The rest of this chapter is the proof — and the professional response.

Why is Kind 3 so different from Kind 2? Two reasons, both visible in the kitchen story:

1. **Behaviour is an infinite object.** A program’s behaviour is its reaction to *every* input — infinitely many of them. Reading the text is finite work (the text is finite); watching the behaviour is infinite work. A guaranteed answer would require examining infinitely many runs, which no algorithm can finish.
2. **The simulation trap.** The most natural way to learn a program’s behaviour is to *run it* — simulate it on an input and watch. But a program may run forever, and the simulator cannot distinguish “still working” from “will never finish”. Any analysis method built purely on running the program can never give a guaranteed answer: it is a *recogniser* (Unit D, Section 4) — it can say “yes” but can never say “no” with certainty. To answer behavioural questions for all programs, a tool would need to *know the future of a computation* — and no algorithm can.

Hold onto Kind 3’s definition; it is the door to everything else. Section 1.4 names the three most important examples (the ones in E.1.1), Section 2 shows how to *prove* undecidability for any new problem, and Section 3 shows the theorem that captures the whole family in one stroke.

1.4 The big three: program equivalence, behavioural identity, and specification satisfaction

The criterion names three exemplars — the most important undecidable problems that arise in real computing. They are the same question in three hats, and each hat is a real product category.

1.4.1 Program equivalence — “do these two programs behave identically?”

The question: given two programs P_1 and P_2, decide whether they *behave identically*: for every input x, P_1 halts with output y exactly when P_2 halts with output y (and both fail to halt on exactly the same inputs). Two programs are **equivalent** if they compute the same function.

Why the world asks it: every refactoring, every rewrite, every “migrate to a new framework”, every “swap the supplier’s scoring engine” project is a bet that the new program is equivalent to the old one. The industry answer today: *tests, golden files, differential testing, and careful review* — run both programs on a large sample of inputs and compare outputs. All of that is evidence, and evidence is valuable — but none of it is a *guarantee*, because no algorithm can give one. The perfect “equivalence checker for all programs” is undecidable (proved in Section 2.4, worked example 4). This is why “the tests pass” is the professional standard for refactoring, and why it always will be.

1.4.2 Specification satisfaction — “does this program meet its specification?”

The question: given a program P and a specification S (a formal statement of what P must do — inputs, outputs, constraints, invariants), decide whether P satisfies S: for every input, P’s behaviour matches what S demands.

Why the world asks it: it is the question of *verification* — the discipline that tries to prove programs correct rather than just test them. Regulators demand it (“prove this system does what its spec says”), safety standards require it (avionics, medical devices, railway signalling), and every software contract is ultimately a specification. The perfect verifier — “read any program and any spec, answer correctly for all of them” — is undecidable (it is Rice’s theorem’s flagship example, Section 3). What exists instead is a family of *scoped* verification tools — model checkers, bounded verifiers, theorem provers — each of which proves correctness for a restricted class of programs or properties (Section 4.4). The honest version of the question is decidable; the universal version is not.

1.4.3 Behavioural identity — “do these two systems produce the same output for every input?”

The question: given two systems (or versions, or vendors’ implementations), decide whether they produce the same output for every possible input.

Why the world asks it: it is the drop-in replacement question. “Can we switch database vendors without changing query results?” “Does the new tax engine agree with the old one on every taxpayer?” “Does this app version behave exactly like the previous one for every user action?” Note that 1.4.1 and 1.4.3 are the same problem under two names — “program equivalence” and “behavioural identity” are synonyms: two programs behave identically exactly when they are equivalent. The study guide lists them separately because industry meets them separately: equivalence is the *refactoring* question (two versions of the same codebase), behavioural identity is the *replacement* question (two different implementations of the same service). The theory treats them as one — and both are undecidable.

1.5 The family portrait — and the family rule

The big three are not three separate theorems; they are three faces of one family. The family rule, worth memorising:

> **The family rule.** Any question of the form “does this program (or do these programs) *behave* like so — for **all** inputs?” — where the answer depends on what the program does when it runs, not on how it is written — is undecidable when asked in its general, all-programs form.

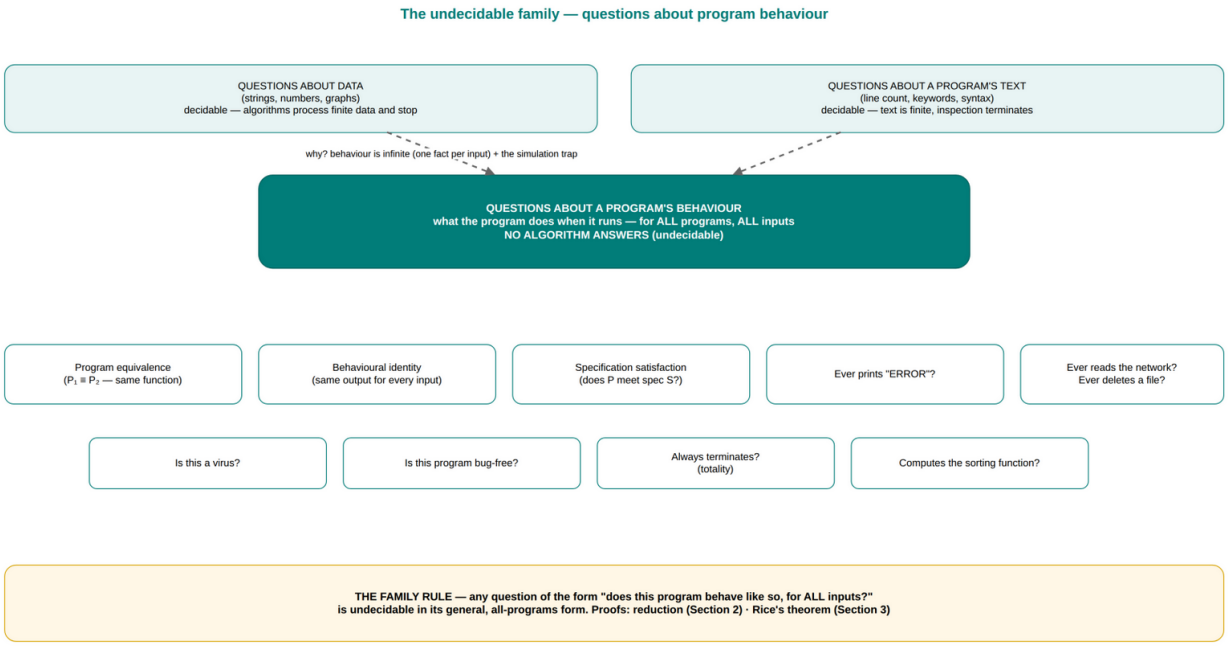


Figure 1 — The undecidable family: questions about program behaviour — equivalence, behavioural identity, specification satisfaction, and more — all live beyond the reach of any algorithm.

Every question in the family shares the same two structural ingredients from Section 1.3: the answer depends on **infinitely many runs** (all inputs), and any attempt to inspect the behaviour directly is caught in the **simulation trap** (a run that never ends). The family is much larger than the big three. Add any of these, and you are still inside it:

- “Does this program ever print `ERROR`?” (every incident-response team’s question — undecidable, proved by reduction in Section 2.4)
- “Does this program ever read from the network?” (the data-exfiltration question — undecidable)
- “Does this program ever delete a file?” (the ransomware question — undecidable)
- “Is this program a virus?” (the antivirus question — undecidable, Section 4.3)
- “Is this program free of bugs?” (the QA question — undecidable, Section 4.3)
- “Does this program always terminate?” (the critical-systems question — undecidable, a direct relative of HALT)
- “Does this program compute the sorting function?” (the correctness-of-a-sort question — undecidable, Section 3.6)

The pattern to internalise: **if the question is about behaviour, and the answer must hold for all programs (or all inputs), it is undecidable.** Section 2 turns this intuition into a proof technique you can apply to any new problem; Section 3 upgrades the pattern into a theorem that covers every case at once.

1.6 Why the whole family is undecidable — the reason in one sentence

It is worth pausing on the *one sentence* that carries the family, because it is the bridge to Sections 2 and 3:

> **A perfect answerer for any behavioural question would be able to predict whether a program halts — because a program can hide its behaviour behind another program's halting.** If a tool could decide "does P behave like X?" for every P, then by wrapping any halting-question (P, x) inside a program whose behaviour switches on P's halting, the tool would decide HALT — which Unit D proved impossible.

That is the reduction idea in its most general form, and Section 2 formalises it into a recipe. Section 3 (Rice's theorem) will show that the argument works for *every* non-trivial behavioural property simultaneously — one proof, the entire family.

Conclusion-first (E.1.1): Problems about *programs* come in two kinds: questions about a program's **text** are decidable (counting lines, checking syntax — linters and scanners do this for a living), but questions about a program's **behaviour** are undecidable in general. The three named examples — **program equivalence** ("do these two programs behave identically?"), **behavioural identity** ("do these two systems produce the same output for every input?" — the same question under the replacement hat), and **specification satisfaction** ("does this program meet its specification?") — are the professional faces of the family: safe refactoring, drop-in replacement and verification. The family rule: any question of the form "does this program behave like so, for all inputs?" is undecidable, because behaviour is an infinite object (every input) and any direct inspection is caught in the simulation trap. Recognising this shape is the first professional skill of this unit — before you can classify a problem (Section 2) or apply the master theorem (Section 3), you must see which questions are behavioural.

1.7 Where you will use this (E.1.1)

- **Evaluating feature requests:** when a stakeholder asks for "a tool that checks whether two versions behave identically", you know immediately that the *universal* version is impossible and the professional version is *differential testing with a documented sample* — you can scope the promise before a line of code is written.
- **Refactoring and migration projects:** "the new code must behave exactly like the old code" is an equivalence claim; the professional instruments are golden tests, property-based tests and code review — never a promise of certainty.
- **Procurement and vendor evaluation:** "does the new supplier's engine produce the same results?" — ask for their equivalence evidence (test suites, conformance suites), not for a guarantee.

- **Verification careers:** the entire field of formal verification exists because the universal verifier is impossible; every job in that field is a job *inside* the scoped, decidable fragments (model checking, bounded verification, verified kernels).
- **Interview depth:** “why can’t we just build a tool that checks whether two programs are equivalent?” is a standard senior interview question; the answer is the family rule plus the reduction sketch of Section 2.4.

1.8 Self-check (E.1.1)

1. State the three kinds of questions about inputs to a computation, and say which kind is undecidable in general — with the two structural reasons.
2. Define program equivalence precisely (what must hold for every input?).
3. Explain, in two sentences each, why program equivalence, behavioural identity and specification satisfaction matter in real industry, and give the professional (non-guaranteed) practice for each.
4. Why is “does this program contain the word `ERROR`?” decidable while “does this program ever print `ERROR`?” is not? Point to the exact difference.
5. State the family rule, and classify each of these as inside or outside the undecidable family, with a one-line reason: (a) “does this string match this regex?” (b) “does this program match this regex behaviour on every input?” (c) “does this program have more than 1 000 lines?” (d) “does this program ever send data to another machine?”
6. (Challenge) A colleague says: “Program equivalence is just testing with enough inputs — we’ll use a million test cases.” Respond using the infinity argument, and say what the million tests are worth.

---

2. Classifying problems: decidable or undecidable — the reduction technique (E.1.2)

> **Assessment criterion E.1.2:** A given problem is classified as decidable or undecidable by applying appropriate techniques (e.g. reduction).

2.1 Scenario: the triage meeting — ten feature requests, one hour

Your team maintains a platform that other teams build on. Every quarter, the product manager brings a list of requests, and your job in the triage meeting is to answer one question for each: *can we build this with a guarantee, or is it an evidence-based feature?*

This quarter’s list:

1. “Validate that a phone number is in the correct SA format.” — surely yes?

2. “Check whether two configuration files are semantically identical (same key–value pairs, order ignored).” — probably yes?
3. “Warn us if any of our 5 000 code files has a `TODO` comment.” — trivially yes?
4. “Guarantee that a new version of the scoring engine behaves exactly like the old one on every possible input.” — ?
5. “Flag any program that will ever read from the network.” — ?
6. “Prove that our scheduling module meets its formal specification.” — ?
7. “Check whether a SQL query finishes within its 2-second budget.” — ?
8. “Detect any code that never terminates, before it reaches production.” — ?
9. “Verify that this sort function sorts every possible list correctly.” — ?
10. “Tell us whether this binary is malware.” — ?

You need a procedure — a repeatable way to classify *any* problem — not a lucky guess. That procedure is the **classification protocol** of this section: a small set of questions you run on any problem statement, which lands you in “decidable” (build it with a guarantee), “undecidable” (build the evidence-based version), or “unknown — research it” (rare, and honest about it). The protocol uses the two techniques of the criterion: **finding a total algorithm** (for the decidable side) and **reduction** (for the undecidable side). By the end of this section, you will have classified all ten items on the list, with proofs.

2.2 Simple analogy: the two baskets and the referee

Tell this to a six-year-old:

A school wants to sort all of its jobs into two baskets. The **Answer basket** holds jobs a robot can finish, always, no matter what: “count how many marbles are in this jar”, “sort these shoes by size”. The robot always finishes, because the job has a definite end. The **Impossible basket** holds jobs no robot can ever finish for sure: “decide whether this marble-rolling maze will ever let the marble out” — the maze can be built with no exit, and the robot would have to roll the marble forever to find out.

The school hires a referee. The referee has one trick, and it settles every argument: **“If a robot could do this job, could it also tell us whether the marble gets out of the maze?”** Whenever the answer is yes, the job goes in the Impossible basket — because nobody, not even the cleverest robot, can answer the maze question for all mazes. For the jobs where the trick fails — the referee cannot build a maze-question out of them — the school checks the other way: does an ordinary, always-finishing recipe exist? If yes, the job goes in the Answer basket.

In grown-up terms: the Answer basket is the **decidable** problems — a total algorithm exists. The Impossible basket is the **undecidable** problems. The maze is the **Halting Problem**. The referee’s trick is **reduction**: to prove a problem X is undecidable, show that a perfect solver for X could solve the Halting Problem (by wrapping any halting question inside an X-question) — and since HALT is impossible, so is X. For the decidable side, the recipe is finding an algorithm and *proving it always terminates*. Two routes, no third: every problem in this section’s scope lands in exactly one basket, and the referee’s trick is the whole art.

2.3 The classification protocol — the procedure, step by step

The protocol is a refinement of Unit D's four-question decidability check, tuned for the E. 1.2 task of classifying a *given* problem. Run it in this order:

Step 1 — State the problem precisely. What exactly is the input, and what exactly is the question? ("Given a program P, does P print the word `ERROR` at any point in any run?" — not "is this code messy?") Precision is not bureaucracy: the reduction technique needs an exact question to wrap, and most misclassifications come from asking the wrong question.

Step 2 — Ask the family question. Is this a question about *data* or about a program's *text* — or about a program's *behaviour* (Section 1.3)? If it is about data or text, it is almost certainly decidable: go to Step 3. If it is about behaviour and the question demands an answer for **all** programs, undecidability is likely: go to Step 4.

Step 3 — The decidable route: find a total algorithm and prove it terminates. An algorithm that always terminates *is* the proof of decidability. Name the algorithm, then give the termination argument: "the input is finite, each step consumes a bounded resource, so the computation stops". The termination argument is the part examiners check — "the regex is simulated on a finite string, and NFA simulation makes one pass per symbol" is a proof; "we just run it" is not (the simulation trap).

Step 4 — The undecidable route: reduce from a known undecidable problem (HALT). Show that if a solver for X existed, it could decide HALT. The reduction recipe, in its standard three moves:

> **The reduction recipe (HALT ≤ X).** Suppose an algorithm decides X. 1. **Take** any HALT-instance (P, x). 2. **Build** a wrapper program P′ (mechanically, from P and x) such that the answer to X's question about P′ is "yes" **exactly when** P halts on x. 3. **Conclude:** the X-decider, fed P′, would answer HALT for every (P, x) — so X cannot be decidable, or HALT would be.

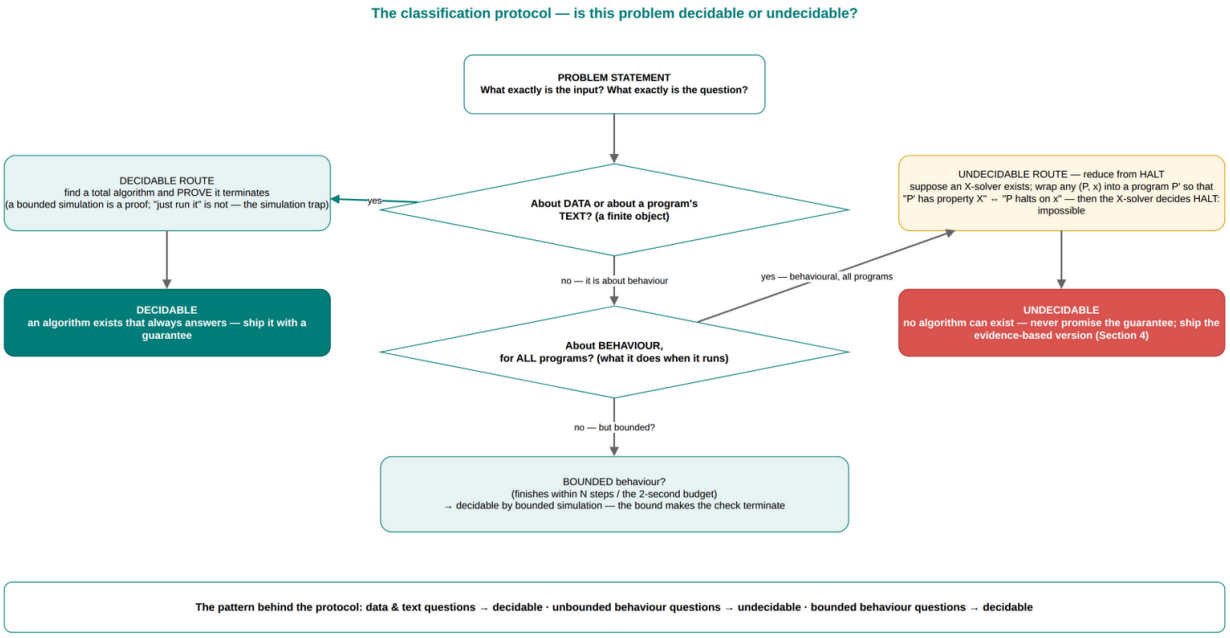


Figure 2 — The classification protocol: state the problem precisely, ask the family question, then take the decidable route (find a total algorithm) or the undecidable route (reduce from HALT).

The wrapper is the heart. The standard trick — used in every reduction in this chapter — is the **delayed trigger**: *make X's property light up exactly at the moment P halts on x.* Concretely: P′ runs P on x first; everything after that first step is the property you want. If P never halts, P′ never reaches the property; if P halts, P′ reaches it. The equivalence "P′ has property X ⟺ P halts on x" then falls out automatically. The unit D notes already proved one reduction with this trick ("does P ever print ERROR?"); Sections 2.4–2.6 run it on a gallery of new problems, including all ten from the scenario.

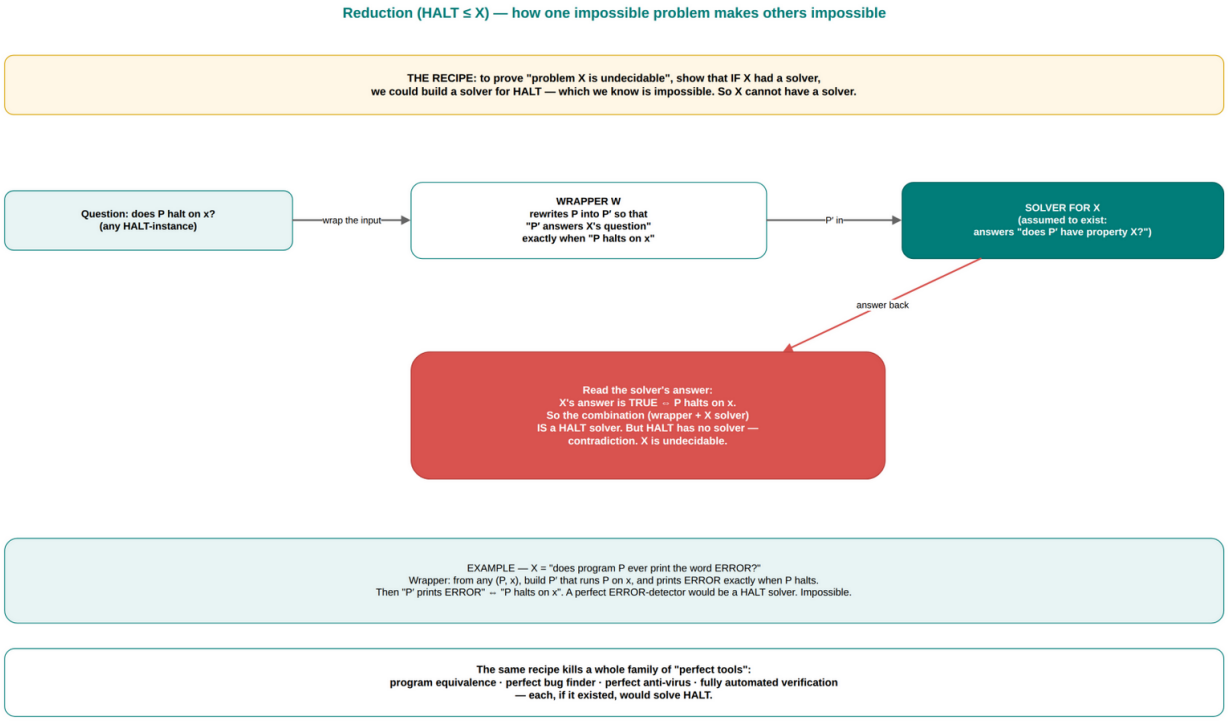


Figure 3 — The reduction recipe: wrap any (P, x) into a P′ whose answer to X's question is "yes" exactly when P halts on x — so an X-solver would decide HALT.

2.4 Worked classifications — the gallery

Now the ten items from the triage meeting, each classified with the protocol. Items 1–3 are the decidable route; items 4–8 walk the reduction route in full; item 9 mixes both; item 10 closes the section's arc. Study the *moves*, not just the answers — the gallery is the exam preparation for every E.1.2 question.

Worked 1 — "Validate this phone number" — DECIDABLE

- **Problem:** given a string, is it a valid SA phone number (e.g. `+27 82 123 4567`)?
- **Protocol:** Step 2 — a question about *data* (a string), not behaviour.
- **Route (Step 3):** a regular expression (Unit F) or a checksum-based validator. **Termination proof:** the input string is finite; the validator examines each character a bounded number of times and then stops. There is no loop whose length depends on anything infinite.
- **Determination: decidable.** (Complexity, Unit C: it is even linear time.)

Worked 2 — "Are these two configuration files semantically identical?" — DECIDABLE

- **Problem:** given two files (key–value sets), do they contain exactly the same pairs, order ignored?
- **Protocol:** Step 2 — a question about *data* (two files).
- **Route (Step 3):** parse both files into sets; compare the sets (sort both, or build a hash set, then compare). **Termination proof:** both files are finite; parsing and set construction touch each entry once and stop.
- **Determination: decidable.** This is the trap-question contrast for the gallery: the *semantic* identity of two **data files** is decidable, while the *behavioural* identity of two **programs** (Worked 4) is not. The difference is exactly the Section 1.3 line: data questions are finite; behaviour questions are infinite. Never confuse the two — examiners love this pairing.

Worked 3 — "Does this file contain a `TODO` comment?" — DECIDABLE

- **Problem:** given a source file, does it contain the string `TODO` in a comment?
- **Protocol:** Step 2 — a question about *text* (Kind 2). Note the contrast with "does this program ever *print* TODO?" — a behaviour question, and undecidable (Section 2.6, Worked 8's shape).
- **Route (Step 3):** scan the file; tokenise; check comments. **Termination proof:** finite file, one pass.
- **Determination: decidable.** This is what linters and style checkers sell: syntactic properties, checked with guarantees.

Worked 4 — "Does this program behave identically to that one?" (program equivalence) — UNDECIDABLE

- **Problem:** given programs P_1 and P_2, do they compute the same function (same output for every input, and both halt or both fail to halt on the same inputs)?
- **Protocol:** Step 2 — a question about **behaviour**, for **all** programs. Undecidability expected.
- **Route (Step 4) — the reduction:** suppose a perfect equivalence-checker E exists. Take any HALT-instance (P, x). Build two wrapper programs:
  - **Z(y)**: the trivial program that, on any input y, prints `0` and stops — it computes the constant function f(y) = 0.
  - **P′(y)**: the wrapper — on any input y, it first runs P on x; **if P halts on x, P′ prints `0` and stops** (so P′ behaves exactly like Z); **if P never halts on x, P′ never stops** (it is stuck in P's run forever, and never prints).

  Now check the equivalence: P′ behaves identically to Z **exactly when** P halts on x.
  - If P halts on x: P′ prints `0` on every input — same function as Z. E answers "equivalent".
  - If P never halts on x: P′ never prints anything — it does not compute f(y) = 0. E answers "not equivalent". So E, fed the pair (P′, Z), answers HALT for every (P, x). **A perfect equivalence-checker would decide HALT — impossible. Program equivalence is undecidable.**
- **The move to remember:** the *delayed trigger* — P′'s identity with A switches on P's halting. The property ("behaves like the trivial program") is attached to halting.

Worked 5 — "Does this program ever print `ERROR`?" — UNDECIDABLE

- **Problem:** given P, does it print the word `ERROR` during any run?
- **Protocol:** Step 2 — behaviour, all programs. Expected: undecidable.
- **Route (Step 4):** suppose a perfect error-printer detector D exists. From any (P, x), build P′: run P on x; **if P halts, P′ prints `ERROR` and stops.** Equivalence: P′ prints `ERROR` ⟺ P halts on x (if P never halts, P′ never prints). So D, fed P′, answers HALT. **Undecidable.**
- **Contrast with Worked 3:** scanning *for the string* is decidable; detecting the *printing* is not. Text versus behaviour — the Section 1.3 line again, now with proofs.

Worked 6 — "Does this program ever read from the network?" — UNDECIDABLE

- **Problem:** given P, does it ever perform a network read during any run?
- **Protocol:** Step 2 — behaviour, all programs.
- **Route (Step 4):** suppose a perfect network-read detector N exists. From any (P, x), build P′: run P on x; **if P halts, P′ performs a network read and stops.**

Equivalence: P′ reads from the network ⟺ P halts on x. So N answers HALT. **Undecidable.**

- **The industrial echo:** this is the data-exfiltration question — "can we guarantee this software never phones home?" The universal guarantee is impossible; what ships instead is *network monitoring, sandboxing with blocked sockets (Section 4.4), and permissions* — all decidable constraints on the *environment*, not on the program's future.

Worked 7 — "Does this program halt on an empty input?" — UNDECIDABLE

- **Problem:** given P, does P halt when run on the empty input ε?
- **Protocol:** Step 2 — behaviour, all programs. (This is HALT with a fixed input — the "empty-input halting problem", HALT-ε.)
- **Route (Step 4):** suppose a perfect empty-input halting detector Hε exists. From any (P, x), build P′: **ignore whatever input you receive; run P on x.** Equivalence: P′ halts on ε ⟺ P halts on x (P′'s behaviour does not depend on its own input at all — it is entirely P-on-x in disguise). So Hε, fed P′, answers HALT. **Undecidable.**
- **The move to remember:** the *input-forgetter* — the wrapper discards its own input and embeds (P, x). This is the standard trick for "fixed-input" variants of HALT (empty input, input `0`, any fixed string).

Worked 8 — "Does this program ever terminate?" (the totality problem) — UNDECIDABLE

- **Problem:** given P, does P halt on *every* input? (A program with this property is called **total**.)
- **Protocol:** Step 2 — behaviour, all programs.
- **Route (Step 4):** suppose a perfect totality detector T exists. From any (P, x), build P′: **on any input y, run P on x; if P halts, P′ halts** (P′'s input is irrelevant — it just runs the embedded computation and stops when that stops). Equivalence: P′ is total ⟺ P halts on x (if P never halts on x, then P′ never halts on *any* y — it is not total; if P halts, P′ halts on every y — it is total). So T answers HALT. **Undecidable.**
- **The industrial echo:** "will this code always terminate?" is the critical-systems question. The universal version is impossible; what ships is *bounded execution* (timeouts, step budgets), *pattern checks* (linters catch obvious loops), and *manual termination proofs for the critical few* (Section 4.4). Note also: this problem is Rice's theorem's first application in Section 3.6 — the same result, one line, once the theorem is in hand.

Worked 9 — "Does this SQL query finish within 2 seconds?" — DECIDABLE (with a nuance)

- **Problem:** given a query and a database, will the query finish within a 2-second budget?

- **Protocol:** Step 2 — this *looks* behavioural, and it is a trap: “within 2 seconds” is a **bounded** behavioural property. A bounded question is finite, because the answer is determined by the first 2 seconds of the computation.
- **Route (Step 3):** run the query with a step budget (or a clock) for exactly the budgeted time. If it finishes, “yes”. If it has not finished, the answer is definitively “no” — it did not finish within the budget, and the first 2 seconds of a computation are a finite, completable object. **Termination proof:** the simulation itself runs for exactly the budget and stops; the bound makes it a total algorithm.
- **Determination: decidable. The nuance:** “2 seconds” as a *step count* is exactly decidable; as *wall-clock time* the guarantee weakens (how fast is the machine? is the cache warm? what else is running?) — the pure question “will it finish within the budget of N steps” is decidable, and the engineering practice (query limits, planner estimates, `EXPLAIN`, statement timeouts) is exactly that decidable question, packaged with a performance model. Compare with the unbounded twin: “does this query finish *at all*?” is the Halting Problem — undecidable. **The bounded/unbounded line is one of the most examinable distinctions in the whole unit.**

Worked 10 — “Is this program a virus?” — UNDECIDABLE

- **Problem:** given a program P, is P malicious — does it behave like a virus (replicates, damages, exfiltrates)?
- **Protocol:** Step 2 — “malicious” is a claim about *behaviour* (“what P does when it runs”), all programs.
- **Route (Step 4):** suppose a perfect virus detector V exists. From any (P, x), build P′: run P on x; **if P halts, P′ performs a clearly malicious act (deletes a file) and stops.** Equivalence: P′ is a virus ⟺ P halts on x. So V answers HALT. **Undecidable.** (Virus detection gets its full treatment in Section 4.3 — this reduction is the skeleton of the whole product category.)
- **The industrial echo:** antivirus products do not claim the impossible — they ship signatures (syntactic, decidable — Worked 3’s technology!), heuristics, sandboxing and monitoring (Section 4.4). “Perfect detection” is impossible; “detect what we know + flag what looks suspicious” is a decidable, profitable industry.

2.5 The decision table — the gallery at a glance

The ten worked examples, in one table you can reuse for classification questions:

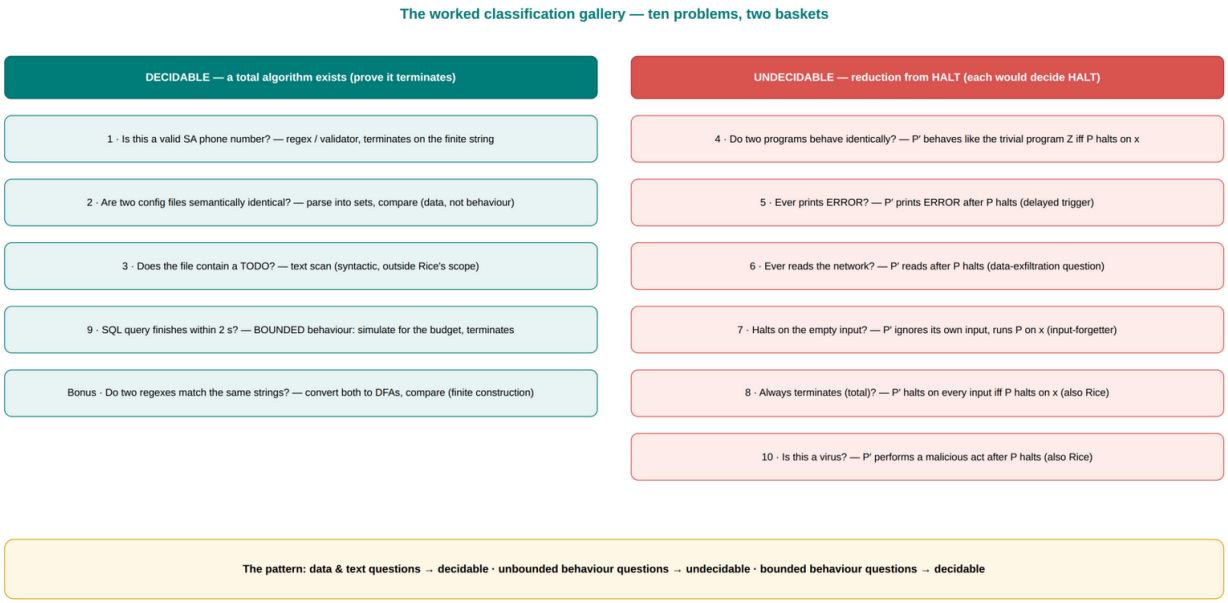


Figure 4 — The worked classification gallery: data and text questions land in the decidable column (total algorithms), unbounded behaviour questions in the undecidable column (reductions), and bounded behaviour questions back in the decidable column.

#	Problem	Kind	Technique	Determination
1	Is this string a valid SA phone number?	Data	Regex/validator, terminates on finite string	**Decidable**
2	Are these two config files semantically identical?	Data	Parse into sets, compare	**Decidable**
3	Does this file contain `TODO`?	Text	Scan, tokenise, check	**Decidable**
4	Do these two programs behave identically (equivalence)?	Behaviour	Reduction: P′ switches on P's halting	**Undecidable**
5	Does this program ever print `ERROR`?	Behaviour	Reduction: print after P halts	**Undecidable**
6	Does this program ever read from the network?	Behaviour	Reduction: read after P halts	**Undecidable**
7	Does this program halt on the empty input?	Behaviour	Reduction: P′ ignores input, runs P on x	**Undecidable**

#	Problem	Kind	Technique	Determination
8	Does this program halt on every input (total)?	Behaviour	Reduction: P′ halts iff P halts on x	**Undecidable**
9	Does this query finish within 2 s?	Bounded behaviour	Bounded simulation, terminates by the budget	**Decidable**
10	Is this program a virus?	Behaviour	Reduction: malicious act after P halts	**Undecidable**

The lesson the table teaches: **data questions and text questions are decidable; unbounded behaviour questions are undecidable; bounded behaviour questions are decidable.** Every row is a proof — a total algorithm, or a wrapper — never a guess.

2.6 The honest shortcuts — recognising the shape without building a wrapper

In an exam you have time for full proofs; in a meeting you do not. Three shape-recognition heuristics let you *suspect* the right answer fast (always back them with a proof when the decision matters):

1. **If the question mentions “program”, “behaviour”, “always”, “ever”, or “for every input” — suspect undecidable.** “Ever”, “always” and “for every input” are the vocabulary of infinity, and infinity is where the undecidable lives.
2. **If the question mentions a finite bound — “within 100 steps”, “first 1 000 lines”, “for inputs up to length 10” — suspect decidable.** A bound makes the computation finite and the answer checkable (Worked 9).
3. **If the question is about the text, the file, or the syntax — decidable by inspection.** Linters, scanners, signature matchers, style checkers: all decidable, all shipped with guarantees.

And the professional version of the third shortcut — the **reduction reflex**: whenever someone claims a perfect tool for a behavioural property, ask the referee’s question: *“Could this tool, fed a wrapper that waits for P’s halting, answer the Halting Problem?”* If yes, the claim is impossible, and the honest product is the evidence-based version. This reflex is worth more than memorising any ten examples.

Conclusion-first (E.1.2): Classifying a problem is a protocol, not a guess: **(1)** state the problem precisely; **(2)** ask the family question (data/text → decidable route; behaviour → undecidable route); **(3)** for the decidable route, name a total algorithm and *prove it terminates* — a bounded simulation is a proof, "we just run it" is not; **(4)** for the undecidable route, run the **reduction recipe** — suppose an X-solver exists, wrap any (P, x) in a program P′ whose X-answer is "yes" exactly when P halts on x, and conclude the X-solver would decide HALT. The **delayed trigger** ("make X's property fire at the moment P halts") and the **input-forgetter** ("P′ ignores its input and runs P on x") are the two wrapper moves that cover nearly every exam reduction. Ten worked classifications show the pattern: data and text questions are decidable, unbounded behaviour questions are undecidable, bounded behaviour questions are decidable. The reduction reflex — "would a perfect tool for this property decide HALT?" — is the professional habit this criterion is really about.

2.7 Where you will use this (E.1.2)

- **The triage meeting (the scenario):** every feature request, bug-report question and vendor claim gets classified in one minute: decidable (scope the algorithm), undecidable (scope the evidence), or bounded (scope the budget). Your team stops over-promising and starts delivering honest features.
- **Architecture decisions:** "should we build an in-house verifier or buy one?" — the answer is decided by classification: if the property is decidable (bounded, syntactic, data), a guaranteed tool is buildable; if not, the purchase is an evidence tool and the contract must say so.
- **Security engineering:** "can we guarantee this binary doesn't phone home?" — the reduction (Worked 6) proves the guarantee is impossible; the professional design is sandboxing + network policy + monitoring, which is exactly what the industry ships.
- **Exam and certification questions:** the worked gallery is the standard question set — "classify: X" appears in assignments, tests and the exam; the protocol + wrapper moves are the full method.
- **Interview depth:** "how would you prove a problem is undecidable?" — the three-move recipe (assume solver, build wrapper, conclude HALT) is a complete, compact answer.

2.8 Self-check (E.1.2)

1. State the classification protocol in four steps.
2. Why is "we run it and watch" not a termination proof? Give the exact failure mode.
3. Restate the reduction recipe in the HALT ≤ X form, and name the two wrapper moves.
4. Classify each of these, with technique and one-line reason: (a) "does this regex match this text?" (b) "do these two regexes match exactly the same strings?" (c) "does this program ever write to a file named `log.txt`?" (d) "does this program halt on the input `0`?" (e) "does this program have fewer than 100 lines?"

5. Build the reduction for “does this program ever write to a file named `log.txt`?” in full — the wrapper, the equivalence, the conclusion.
6. Build the reduction for “do these two programs behave identically?” in full, using the trivial program Z from Worked 4.
7. (Challenge) Why is “finishes within 2 seconds” decidable while “finishes eventually” is not? Give the bounded/unbounded argument precisely, and explain where the wall-clock nuance enters.

---

3. Rice’s theorem — one proof for every behavioural property (E.1.3)

> **Assessment criterion E.1.3:** Rice’s theorem is stated and applied to show that no algorithm can decide any non-trivial property of a program’s behaviour.

3.1 Scenario: the CEO who wants the one tool that answers everything

The triage meeting went well — you classified ten problems. Then the CEO arrives with the natural follow-up question, and it is a good one: *“Why do we keep proving these one at a time? Equivalence, totality, virus-ness, specification satisfaction — they’re all the same shape. Can’t we just prove ONCE that every behavioural question is impossible, and then stop?”*

That question has an answer, and it is one of the most beautiful theorems in computer science: **Rice’s theorem** (Henry Gordon Rice, 1953). It says, in one sentence: *every non-trivial question about a program’s behaviour is undecidable.* Not “these ten” — every question of that kind, including ones nobody has ever thought of. The CEO is right: one proof covers them all, and after this section you will never need a per-property diagonalisation again. You will instead do what working theorists and examiners do: state the problem, check that it is a non-trivial behavioural property, and write “undecidable by Rice’s theorem” — with the two-line verification that it qualifies.

3.2 Simple analogy: the snail messenger

Tell this to a six-year-old:

A king wants to hire a magical judge who can look at any animal and answer one question about it: *“Does this animal sing?”* The judge is very good. But the king’s wizard explains why no such judge can ever exist.

The wizard can build an animal that does not sing until something happens: a snail is sent on a journey, and the animal sings only if — and only when — the snail returns. Nobody

knows whether the snail will ever come back; it might wander forever. Now watch the judge. If the judge says “this animal sings”, the animal has not sung yet, and will sing only if the snail returns — the judge cannot know. If the judge says “this animal does not sing”, the snail might be about to arrive — the judge cannot know that either. To answer correctly, the judge would have to know *whether the snail returns* — and nobody can know that, because some journeys never end.

And here is the wizard’s trick: **this works for any question at all.** Ask “does this animal dance?” — the wizard builds an animal that dances only after the snail returns. Ask “does this animal paint pictures?” — same trick. The snail can be attached to *any* interesting behaviour, so no judge can ever be trusted on *any* interesting question about an animal.

In grown-up terms: the animal is a *program*; “sings”, “dances”, “paints” are *behavioural properties*; the snail is *the halting of another program*; the judge is a *decider for the property*. Rice’s theorem says: for **any** property P of a program’s behaviour that is **non-trivial** (at least one program has it and at least one does not), a decider for P could be turned into a decider for the Halting Problem — because we can always build a program whose P-behaviour is switched on by another program’s halting (the snail). The snail is attached to every property, so every property is undecidable. One wizard, every question.

3.3 From first principles: what “a property of a program’s behaviour” means

To state Rice’s theorem precisely — and to apply it safely — three definitions must be built from nothing.

Definition 1 — Behaviour is the function computed. A program P, on input x, either halts with an output y or runs forever. Write this as a function: P computes the **partial function** f: for inputs where P halts, f(x) = the output; for inputs where P runs forever, f(x) is *undefined*. “Behaviour” is exactly this partial function — the complete input–output relation. Two programs with the same partial function behave identically; that is precisely the equivalence notion of Section 1.4.

Definition 2 — A behavioural property is a set of behaviours. A property of behaviour is a yes/no question answered by the function, not by the text: “is the function total?” “is f(1) = 42?” “does the function sort?” “is the function the identity function?” Each property P is a collection of partial functions — the ones that have the property. A program has property P if its computed function is in that collection.

Definition 3 — Non-trivial means “not everything, not nothing”. A property P is **trivial** if either *every* program has it or *no* program has it. (“Computes some function” — every program computes a partial function, so that property is trivial-true. “Computes a function whose graph is finite” — some do, some don’t: non-trivial.) A property is **non-trivial** if there is at least one program with it and at least one program without it. Non-triviality is the whole gate: Rice’s theorem applies exactly to the non-trivial properties, and the trivial ones are — trivially — decidable (the always-true property is decided by the program that prints “yes”; the always-false by the program that prints “no”).

One more distinction, and it is the one that keeps students out of trouble: **behavioural (semantic) properties** (about the function: totality, correctness, “outputs 42 for input 1”) are Rice’s territory. **Syntactic properties** (about the text: “fewer than 100 lines”, “contains the string `TODO`”) are not properties of the function at all — they are decidable by inspection (Worked 3 in Section 2.4), and Rice says nothing about them. And there is a third category that looks behavioural but escapes the theorem: **bounded behavioural properties** (“halts within 100 steps”) are decidable by bounded simulation (Worked 9) — Rice’s theorem covers properties of the *function*, and a function has no “steps”. Section 3.7 maps these boundaries fully, because applying a theorem safely means knowing exactly where it stops.

3.4 Rice’s theorem — stated

With the three definitions in place, the theorem fits in two lines:

> **Rice’s theorem.** Let P be any non-trivial property of the behaviour of programs (any non-trivial property of the partial function computed by a program). Then the problem “given a program, does it have property P?” is **undecidable**.

Unpack each word: **non-trivial** — some program has P, some does not (Definition 3). **Property of behaviour** — P is determined by the computed partial function, not by the text (Definitions 1–2). **Undecidable** — no algorithm answers “does this program have P?” for all programs, ever.

Notice what the theorem does *not* say: it does not say the property is “hard to check” or “unknown”; it says **no algorithm exists**. It does not depend on how clever the property is, how obscure the programs are, or how fast the hardware becomes. And it is **constructive** in the strongest sense: for any claimed solver, the proof (Section 3.5) hands you a specific program it fails on — exactly like the diagonal proof of HALT in Unit D. No future cleverness escapes.

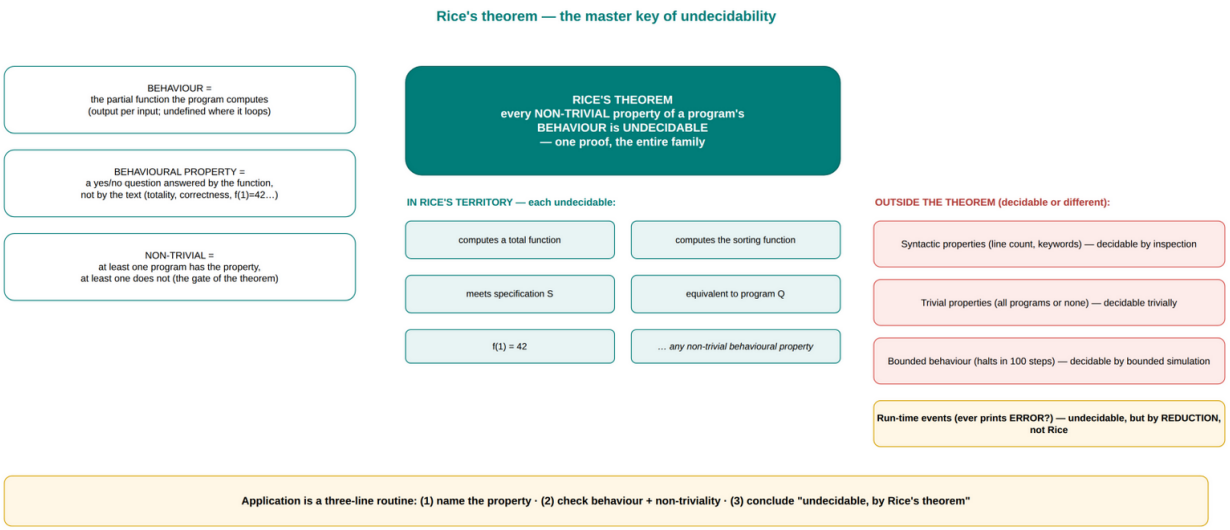


Figure 5 — Rice's theorem: every non-trivial behavioural property — totality, correctness, equivalence to a function, anything — is undecidable; only syntactic, trivial and bounded questions escape.

3.5 The proof — the snail in formal dress

The proof is a reduction, and it is the cleanest reduction in the module. It uses the snail exactly as the story did.

Theorem. Let P be a non-trivial behavioural property. Then "does this program have P?" is undecidable.

Proof. Since P is non-trivial, choose a witness: a program **A** that *has* property P. (Such a program exists — that is the first half of non-triviality.) Suppose, for contradiction, that an algorithm D decides P: given any program, D says "yes" or "no" correctly about whether it has property P.

Now take any HALT-instance (P_0, x_0). Build the wrapper W:

> W(y): first run P_0 on x_0. If P_0 halts on x_0, then behave exactly like A on y (do whatever A does with y). If P_0 never halts, W never reaches the A-behaviour — it is stuck inside P_0's run forever.

The construction is legitimate: W is a real, writable program — it runs another program inside itself, exactly as every runtime, VM and container does. Note what the wrapper does *not* do: it never decides "P_0 won't halt" — it has no else-branch. It simply tries to run P_0 first, and the A-behaviour happens only *after* that run ends. The halting of P_0 is therefore wired *structurally* into W's behaviour: the snail's arrival is the only thing that lets W ever sing.

Now observe the equivalence, the snail arriving:

- **If P_0 halts on x_0:** W eventually reaches the A-behaviour, so W computes the same function as A — **W has property P** (because A has it).
- **If P_0 never halts on x_0:** W never reaches the A-behaviour — it never produces any output on any input — so W computes the **nowhere-defined function ⊥** (undefined on every input).

The first branch is perfect. The second branch is where the proof must pause: we know W ≡ ⊥, but do we know whether ⊥ has property P? Not yet — and the answer splits the proof in two.

Case 1 — ⊥ does not have P. Then the two branches are exactly what we want: W has P ⟺ P_0 halts on x_0. So D, fed W, answers HALT for every (P_0, x_0) — D decides the Halting Problem. Contradiction with Unit D. Done.

Case 2 — ⊥ has P. Then the wrapper above is useless: W has P in *both* branches, so D would just answer "yes" always — no information about P_0's halting. The fix uses the *second* half of non-triviality: because P is non-trivial, there is also a program **C** that *lacks* P. Build the twin wrapper:

> W′(y): first run P_0 on x_0. If P_0 halts on x_0, then behave exactly like C on y.

Now the branches flip: if P_0 halts, W′ ≡ C, which lacks P — W′ does **not** have P. If P_0 never halts, W′ ≡ ⊥, which *has* P (this case's assumption). So: W′ has P ⟺ P_0 never halts on x_0 — and D, fed W′, answers the *negation* of HALT (it says "has P" exactly when P_0 fails to halt). Deciding the negation of HALT is exactly as impossible as deciding HALT itself — a negated solver could answer HALT by flipping its output. Contradiction. Done.

Either way — Case 1 or Case 2 — a decider for P would decide (a version of) the Halting Problem, which Unit D proved impossible. The assumption "D exists" always collapses. **No decider for P exists — Rice's theorem. ∎**

That was the only place in this chapter where the proof is genuinely delicate — and the payoff is enormous. The two-case construction (use the P-witness first; if the nowhere-defined function itself has P, switch to the non-witness C and read the answer inverted) is the *standard* Rice proof, and examiners who ask for it want exactly this shape. For applying the theorem — the criterion's real requirement — you will never need the proof again: you check non-triviality and behaviour, and cite.

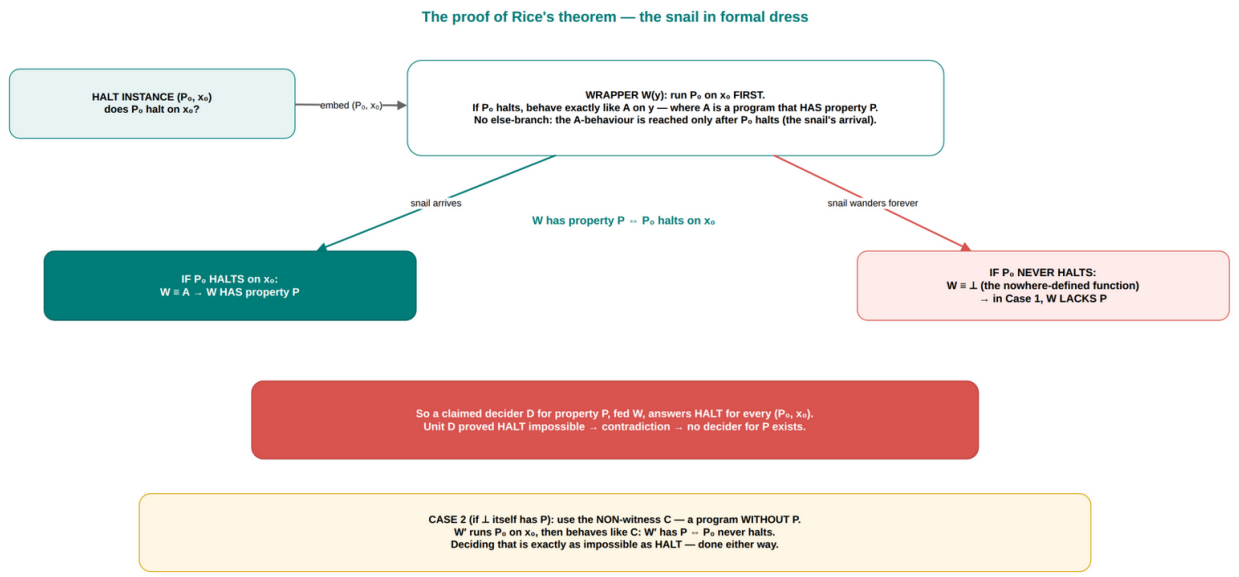


Figure 6 — The proof of Rice's theorem: the wrapper W runs P_0 on x_0 first and only then behaves like A — so "W has property P" is switched on by "P_0 halts on x_0".

3.6 Applying Rice's theorem — the instant results

This is the criterion's core skill: *state the theorem and apply it.* The application is a three-line routine:

1. **Name the property.** "Does this program compute the sorting function?"
2. **Check behaviour, check non-triviality.** Behaviour: yes — "computes the sorting function" is a claim about the function computed. Non-trivial: yes — some programs

compute the sorting function (any correct sort) and some do not (a program that returns the input unchanged; a program that loops forever on one input).

3. **Conclude.** Undecidable, by Rice's theorem.

The routine kills the whole family in seconds — every row of this table is a property P, and every row is undecidable by the same three lines:

Property P (non-trivial behavioural)	**Real-world echo**	**Determination**
"The program computes a total function (halts on every input)"	Critical-systems termination guarantees	**Undecidable** (Rice)
"The program computes the sorting function"	"Verify my sort is correct"	**Undecidable** (Rice)
"The program computes the identity function"	API conformance	**Undecidable** (Rice)
"f(1) = 42 for the computed function"	One-case behavioural contracts	**Undecidable** (Rice)
"The program satisfies specification S" (S given)	Verification, certification	**Undecidable** (Rice)
"The program is equivalent to a given program Q"	Refactoring, replacement	**Undecidable** (Rice)
"The program computes a function that is correct on all inputs"	Complete bug detection	**Undecidable** (Rice)
"The program computes a prime number for every input"	Guaranteed-output contracts	**Undecidable** (Rice)

Check the two hardest rows by hand, because they are the ones that teach the routine. **"Satisfies specification S":** behaviour — S describes what the program must *do* (a property of the computed function); non-trivial — some programs meet S (a correct implementation) and some do not (a program that immediately crashes on every input). **"Equivalent to a given program Q":** behaviour — equivalence is equality of computed functions; non-trivial — Q itself has the property, and a program that loops forever on every input does not (unless Q is that program — and if Q is the nowhere-defined program, pick another non-witness; non-triviality guarantees one exists). Both fall to the three lines.

And the flagship application, which the criterion's wording intends you to meet: **specification satisfaction (VERIFY).** "Does program P satisfy specification S?" is the perfect-verifier question of Section 1.4.2. With Rice's theorem it takes one line: for any fixed non-trivial S, "meets S" is a non-trivial behavioural property, hence undecidable — the universal verifier cannot exist. (This is the theorem behind Section 4's "fully automated verification is impossible".)

3.7 What Rice's theorem does NOT say — the boundaries

A theorem is only as safe as its boundaries, and applying Rice's theorem wrongly is a classic exam error. The theorem covers **non-trivial properties of the computed function**. Three kinds of questions stand outside it:

Boundary 1 — Syntactic properties (about the text) are decidable. "Does this program have fewer than 100 lines?", "does it use the `goto` keyword?", "does its source contain `ERROR`?" — these are questions about the string, not the function, and they are decidable by inspection (Section 2.4, Worked 3). Linters, style checkers and — crucially — **antivirus signature matchers** live exactly here: a signature is a syntactic pattern, so signature matching is decidable, even though "is this a virus?" (behavioural) is not. The decidable island inside the undecidable sea is where the whole pattern-matching industry lives.

Boundary 2 — Trivial properties are decidable. "Does the program compute *some* function?" — every program does, so the answer is always "yes" (decide it with the program that prints "yes"). "Does the program compute a function different from itself?" — a convoluted way of saying "always yes" again. Trivial properties are the empty set of Rice applications: non-triviality is the gate, and checking it is the first line of every application.

Boundary 3 — Bounded behavioural properties are decidable. "Halts within 100 steps", "outputs before the 2-second budget", "uses fewer than 10 MB in the first minute" — these are properties of the *computation prefix*, not of the function, and they are decidable by bounded simulation (Section 2.4, Worked 9). This is why CI timeouts, step limits and resource budgets are legitimate, guaranteed engineering: they test a bounded, decidable property. Note the subtle line: "halts within 100 steps" is decidable; "halts" is not; the difference is exactly the bound.

There is one more subtlety that separates careful students from careless ones, and it is worth stating explicitly: **Rice's theorem is about the function computed, not about incidental run-time events.** "Does the program ever print `ERROR`?" is a behavioural-looking question that Rice does *not* directly cover — printing is an event of the run, not a property of the computed function (a function has no output side-effects in the model). Such questions are still undecidable — but by the *reduction* route of Section 2.4 (Worked 5), not by Rice. In an exam, the complete answer to "classify: does this program ever print ERROR?" is: "undecidable — not by Rice (printing is not a function property), but by reduction from HALT: run P on x, print ERROR iff P halts." The ability to say *which* theorem handles *which* question is exactly the E.1.2/E.1.3 boundary, and examiners deliberately probe it.

The boundaries, in one picture — use it as the safe-application map for the theorem:

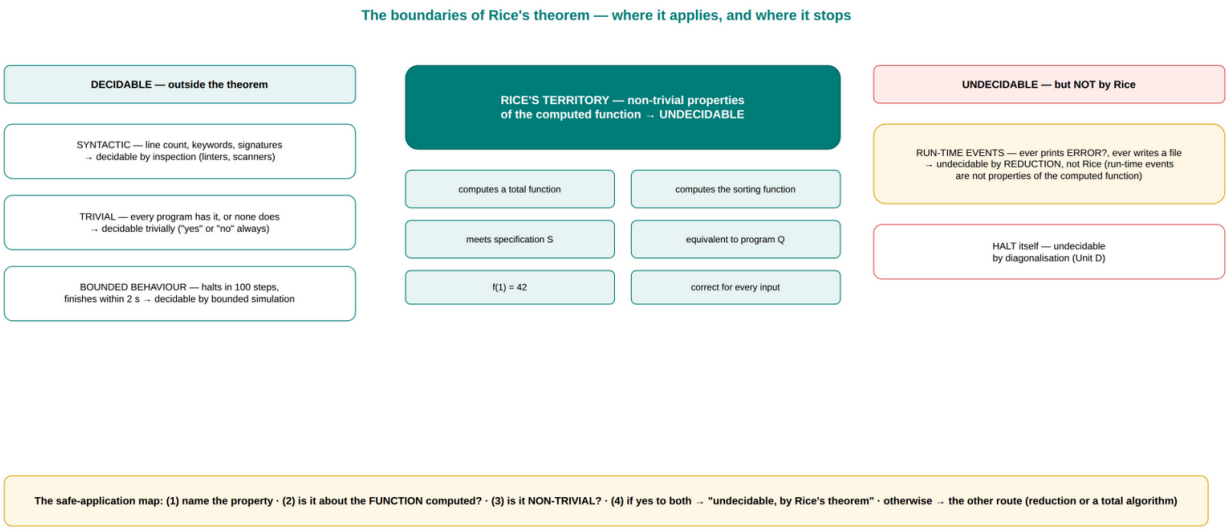


Figure 7 — The boundaries of Rice's theorem: inside (non-trivial behavioural properties — undecidable), and outside (syntactic, trivial and bounded properties — decidable; run-time events — undecidable by reduction).

Conclusion-first (E.1.3): Rice's theorem is the master key of undecidability: *any* ***non-trivial property of a program's behaviour*** *— any non-trivial property of the partial function it computes — is undecidable.* Three definitions carry it: behaviour is the function computed; a behavioural property is a set of functions; non-trivial means some program has it and some does not. The proof is one reduction: choose a program A with the property, build the wrapper W that runs P_0 on x_0 and only then behaves like A — then "W has the property" is switched on by "P_0 halts on x_0", so any property-decider decides HALT (with the two-case fix when the nowhere-defined function itself has the property). **Application is a three-line routine** — name the property, check behaviour + non-triviality, conclude — and it settles totality, correctness of a sort, equivalence, specification satisfaction, and every row of the behavioural family in one stroke each. **Boundaries matter:** syntactic properties, trivial properties and bounded behavioural properties are decidable and outside the theorem; and incidental run-time events (printing, file access) are undecidable by *reduction*, not by Rice. Knowing what the theorem covers, and what it does not, is the difference between applying it and misapplying it.

3.8 Where you will use this (E.1.3)

- **Instant classification:** any "can we build a tool that checks behaviour X for all programs?" question becomes a 30-second Rice check — name the property, test non-triviality, conclude. You stop re-proving HALT for every feature request.
- **Vendor claim audits:** "our tool verifies that programs meet specifications" — Rice says the universal version cannot exist; the procurement question becomes "what class of programs and specs does it cover?" (Section 4.5).

- **Product design:** when the Rice check says “undecidable”, the design reflex switches to the decidable islands — syntactic checks, bounded checks, scoped verification (Section 4.4). The theorem literally guides architecture.
- **Safety and compliance conversations:** “prove this system meets its spec” — the honest answer names the scope; Rice explains *why* the scope is not a concession but the only honest form.
- **Exams:** “state and apply Rice’s theorem” is a near-certain E.1.3 question; the routine of 3.6 plus the boundaries of 3.7 are the complete answer.

3.9 Self-check (E.1.3)

1. State Rice’s theorem, defining “behaviour”, “property of behaviour”, and “non-trivial” as you go.
2. Why must a property be non-trivial for Rice to apply? Give an example of a trivial property and say how it is decided.
3. Sketch the proof: choose A with the property, build W, state the equivalence, and explain where the contradiction bites. Where does the two-case construction come in, and what is the nowhere-defined function’s role?
4. Apply the three-line routine to: (a) “does this program compute a total function?” (b) “does this program compute the identity function?” (c) “does this program satisfy specification S?” (d) “does this program ever print `ERROR`?” — and for (d), say why Rice does not apply directly and what does.
5. Classify each as inside or outside Rice’s scope, with a one-line reason: (a) “fewer than 100 lines” (b) “halts within 100 steps” (c) “computes a function that is 0 on all inputs” (d) “contains the string `TODO`” (e) “is equivalent to program Q”.
6. (Challenge) A colleague says: “Rice’s theorem says we can’t detect viruses, but our antivirus catches thousands a day.” Respond precisely: what is decidable about their product, and what guarantee are they actually selling?

---

4. The practical implications — why perfect tools do not exist, and what industry does instead (E.1.4)

> **Assessment criterion E.1.4:** The practical implications of undecidability are explained — why perfect virus detection, complete bug detection and fully automated verification are impossible, and why industry relies on testing, analysis tools and heuristics.

4.1 Scenario: the impossible promises meeting

Three meetings, three promises — and all three are broken:

Meeting 1 — the antivirus vendor. The security budget is on the table. The vendor's slide says: *"Our product detects 100% of malware, guaranteed."* Your CISO, who has studied this module, asks the one question: "what is the detection rate on malware your signatures have never seen?" The vendor hesitates. The honest answer, as you now know, is: no product can guarantee detection of all malware — because "is this program a virus?" is a non-trivial behavioural property, and Rice's theorem (Section 3.6) says no algorithm decides it. The vendor's "100%" is either a claim about *known* malware (true but not what the slide says) or a lie.

Meeting 2 — the QA promise. The release meeting: *"Can we guarantee the new release is bug-free?"* The honest answer is no — "free of bugs" means "correct on every possible input", a non-trivial behavioural property (Section 3.6: "correct on all inputs" is Rice-undecidable). What the team actually has: tests, fuzzing, code review and a deadline.

Meeting 3 — the regulator. *"Prove that the patient-booking system meets its specification."* The universal version — an automated verifier that reads any program and any specification — is Rice-undecidable (Section 3.6's flagship row). What the company actually does: model checking on the finite state space, verified builds of critical components, and a sign-off document that names its scope.

In all three meetings, the professional is the one who says: *"The universal guarantee is impossible — here is the theorem — and here is the honest, scoped version we can deliver, with its exact guarantee."* That is the entire content of E.1.4, and the entire difference between engineering and marketing.

4.2 Simple analogy: the bridge inspector

Tell this to a six-year-old:

A town needs a bridge across the river, and the mayor wants a promise: *"Can you guarantee the bridge will never fall down?"* The bridge builder is honest: nobody can guarantee that. A storm nobody has imagined, a truck heavier than any ever built, metal that weakens silently over fifty years — the future is too big to check, and some futures are unending. If the builder promised "never falls", the promise would be empty.

So the builder does something better: she promises what is *provable*. She tests the concrete: sample blocks are crushed in the lab until they break — that tells her exactly how strong the material is. She load-tests the finished span with the heaviest trucks the law allows. She inspects every weld, measures every cable, and installs sensors that watch for cracks forever after. She sets a weight limit and publishes it. The guarantee she signs is precise: *"This bridge carries the legal load under the design conditions, with safety factor 3, and sensors will warn us if it changes."* The town accepts — because a scoped, evidenced, monitored guarantee is worth more than an empty "never".

In grown-up terms: the bridge is a *program*; "never falls down" is *perfect correctness on all inputs* — Rice-undecidable; the crush tests are *testing on samples*; the load test is *testing under realistic conditions*; the sensors are *monitoring and observability*; the weight limit is *enforced constraints and budgets*. The professional does not mourn the impossible guarantee — she engineers the strongest guarantee that exists: bounded, scoped,

evidenced, monitored. That is the entire industry practice this section describes: testing, analysis tools and heuristics are not “fallbacks for a failed theorem” — they are the correct engineering of a world in which perfect tools cannot exist.

4.3 The three impossible products — and their exact theorem

The criterion names three products. Each one is a theorem wearing a business suit:

Product 1 — Perfect virus detection: impossible. “Is this program a virus?” is a question about behaviour (what the program does when it runs: replicates, damages, exfiltrates) — and it is non-trivial: some programs are viruses and some are not. **Rice’s theorem: undecidable** (Section 3.6’s family; the direct reduction was Worked 10 of Section 2.4). No algorithm detects all malware, ever — including zero-day malware. The direct reduction is worth remembering in interviews: a wrapper that runs P on x and, if P halts, performs a clearly malicious act — a perfect detector would decide HALT.

Product 2 — Complete bug detection: impossible. “Is this program free of bugs?” — read precisely, “is this program correct for every input?” — is a property of the computed function (correctness relative to its intended behaviour), and it is non-trivial: a correct implementation has it, a program that crashes on input 1 does not. **Rice’s theorem: undecidable.** This is why “bug-free” is not a quality claim but a category error, and why the industry’s actual promises are scoped: “no known bugs on the tested input distribution”, “all reported issues fixed”, “fuzz-tested for N hours with no crash”.

Product 3 — Fully automated verification: impossible. “Read any program and any specification; prove whether the program meets the spec” — for a fixed specification S, “meets S” is a non-trivial behavioural property (Section 3.6’s flagship row). **Rice’s theorem: undecidable.** No universal verifier exists. What exists — and is sold, certified and relied upon — is *scoped verification*: prove a *particular* program against a *particular* specification, using techniques that exploit finiteness (model checking a finite state space), or bounds (bounded verification for N steps), or human-guided proof (interactive theorem proving), or verified compilers and kernels (seL4, CompCert — Section 4.4). The scopes are why the industry survives the theorem: each tool answers a *smaller, decidable* question, and the product documents which one.

4.4 The evidence-based toolkit — what industry actually ships

Here is the professional reframe: undecidability does not mean “nothing can be done” — it means *“the guarantee must be scoped, and the promise must be honest.”* Industry ships nine families of replacements, each with its own exact guarantee. Know them by name, because they are the answer to every “but what do we DO?” question:

Impossible promise	What industry ships	The honest guarantee	Why it works (the theory)
Perfect virus detection	**Signatures & pattern matching** (known malware: hashes, byte patterns, behavioural fingerprints)	“Detects everything in our signature database, immediately”	*Syntactic* matching is decidable (Section 3.7, Boundary 1) — the decidable island
Perfect virus detection	**Heuristics & scoring** (suspicious behaviour: packed binaries, unusual API calls, sandbox behaviour)	“Flags suspicious programs with a documented score; some false positives”	*Bounded simulation* in a sandbox is decidable (Section 2.4, Worked 9) — plus statistical judgment
Complete bug detection	**Testing & fuzzing** (unit, integration, property-based, fuzz with random/ adversarial inputs)	“No known bug on the tested input distribution”	*Sampling* — evidence about actual runs, not proof about all runs (Section 1.3)
Complete bug detection	**Static analysis** (data-flow, null-checks, taint analysis, linters)	“Catches every instance of the *pattern class* it models; some false alarms (over-approximation)”	Checks a *restricted, decidable* model of the program — never the full behaviour
Complete bug detection	**Code review & process** (human review, standards, pairing)	“Expert inspection of the code as written”	Human analysis, outside the theorem’s scope
Fully automated verification	**Model checking** (exhaustively explore the *finite* state space: protocols, hardware, finite workflows)	“Every reachable state of the model satisfies the property”	*Finite* state space → exhaustive check terminates — decidable (Section 2.4, Worked 9’s principle)
Fully automated verification	**Bounded verification** (prove correctness for runs up to N steps / N memory cells)	“Correct for every run of length ≤ N”	*Bounded* property → decidable by construction
Fully automated verification	**Verified artefacts** (seL4 kernel, CompCert compiler — one fixed program, one fixed proof)	“This *specific* program provably meets *this specific* specification”	One program, one proof — finite work about a fixed artefact (Section 3.7, Boundary context)

Impossible promise	What industry ships	The honest guarantee	Why it works (the theory)
All of the above	**Timeouts, budgets, watchdogs** (CI timeouts, query limits, resource caps, health checks)	“Any job that exceeds its budget is killed and reported”	*Bounded* behavioural checks are decidable (Worked 9); the recogniser is converted into a bounded, reportable process (Unit D)

Read the “why it works” column carefully: **every row is the theory, not an exception to it.** Signatures are decidable because they are syntactic. Sandboxes and timeouts are decidable because they are bounded. Model checking is decidable because the state space is finite. Testing is evidence because sampling is not proof. The industry does not defeat Rice’s theorem — it navigates around it, row by row, on the decidable islands the theorem’s own boundaries mark out.

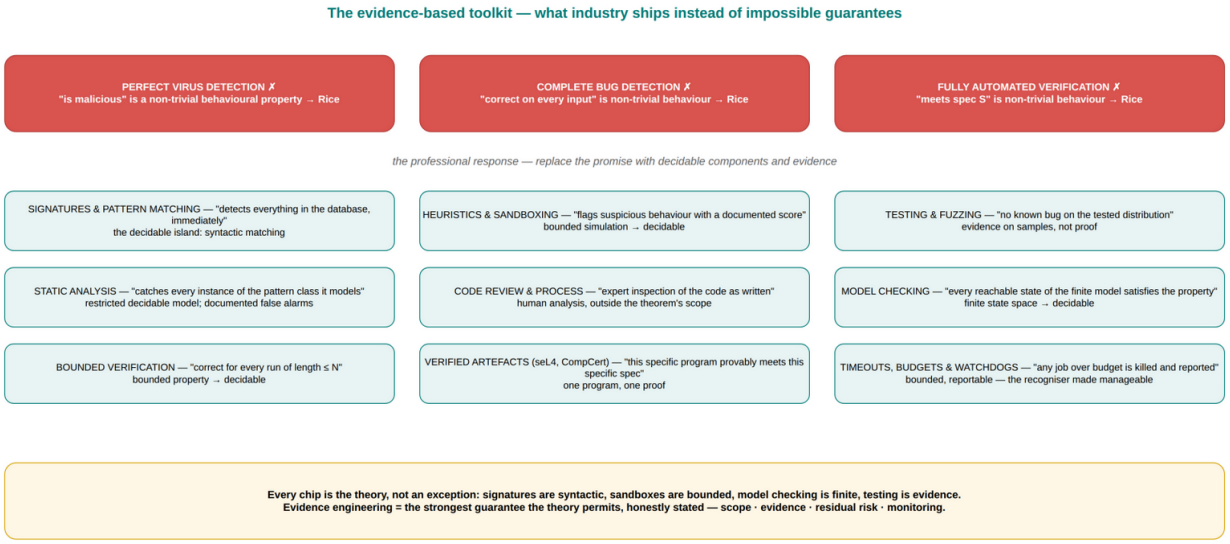


Figure 8 — The evidence-based toolkit: each impossible promise is replaced by decidable components (signatures, bounded simulation, finite-state checks) and evidence methods (testing, analysis, review), each with its own honest guarantee.

4.5 The trust ladder — what you can honestly promise

The final skill of this unit is the hardest and most valuable: *converting an impossible promise into an honest contract.* The professional’s answer to “can you guarantee X?” is never “yes” (a lie) and never “no” (a surrender) — it is the **trust ladder**:

1. **Name the scope.** “We guarantee X *for this class of programs, this class of properties, this bound.*” (The bridge’s weight limit.)
2. **Name the evidence.** “The guarantee is backed by: exhaustive tests on this distribution, N hours of fuzzing, this static analysis with these documented false alarms, this model check of the finite model, this review process.”

3. **Name the residual risk.** “What we do NOT guarantee: behaviour outside the tested distribution, properties outside the checked class, unbounded runs.”
4. **Name the monitoring.** “And what protects us live: timeouts, budgets, sensors, alerting — the bridge’s crack sensors.”

Applied to the three meetings of Section 4.1: the antivirus purchase becomes “detection of all signatures + sandboxed heuristic scoring + incident response”, with an SLA that says so; the QA promise becomes “no known bugs on the tested distribution, all reported issues fixed, fuzz campaign of 48 h clean”; the regulator’s proof becomes “model-checked finite booking model against the safety invariants, bounded verification of the scheduling horizon, signed with the scope and residuals”. Each is a guarantee that can be *kept* — which is worth more than the guarantee that cannot.

The reverse skill — **interrogating a vendor’s guarantee** — is the same ladder upside down. When a supplier claims “100% detection” or “proves any program”, ask: (1) which class of inputs? (2) which properties are checked exactly and which approximately? (3) what is the false-positive and false-negative policy? (4) what is excluded? (5) what is monitored at runtime? If the vendor cannot answer, the guarantee is marketing; if they can, the product is honest engineering — and you now know exactly which questions to ask, because the theory tells you which answers are possible.

4.6 The professional synthesis — evidence engineering

Step back and the unit’s map is complete:

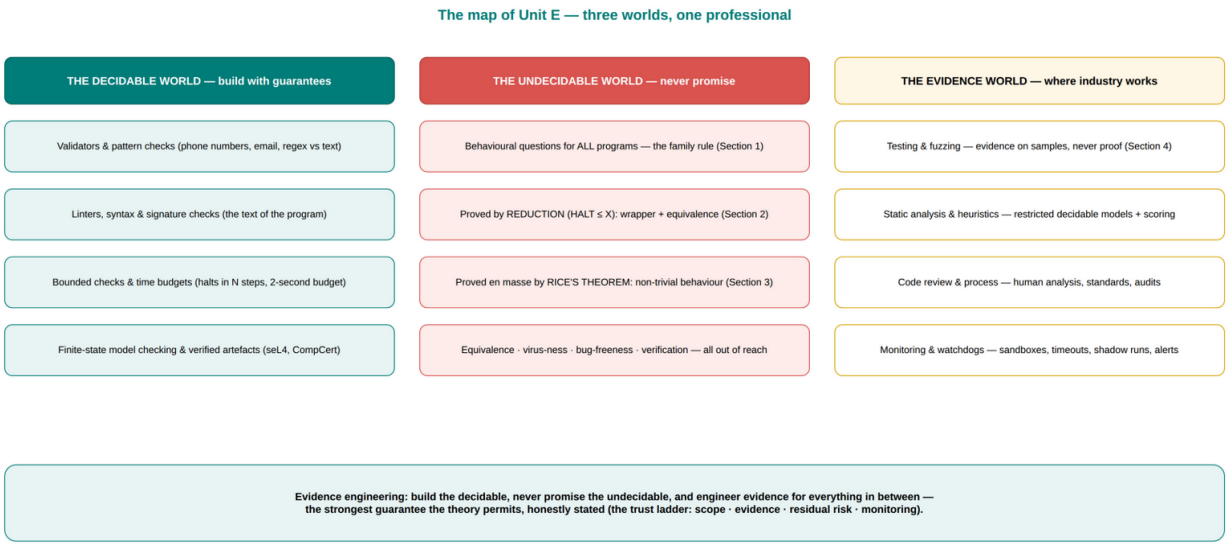


Figure 9 — The map of Unit E: the decidable world (data, text, bounded questions), the undecidable world (behavioural questions, all provably out of reach), and the evidence world where industry actually works — testing, analysis, heuristics, scoped verification.

Three worlds, and the professional lives in all three: **build** the decidable (validators, linters, bounded checks, finite-state model checking — with guarantees), **avoid** promising the undecidable (universal behavioural checks — with the theorem as your reason), and **engineer** the evidence world for everything the decidable cannot reach (testing, fuzzing,

static analysis, heuristics, monitoring, review — with scoped, honest contracts). This is **evidence engineering**: the discipline of producing the strongest guarantee the theory permits, and of saying exactly what it covers. It is not a consolation prize for a failed theorem; it is the reason the software industry functions at all — every product you have ever used is a bundle of decidable checks and evidence-based judgments, wrapped in an honest-enough contract.

> **Conclusion-first (E.1.4):** Undecidability is not a theoretical curiosity — it is the boundary of every product promise. **Perfect virus detection** ("is this a virus?" is a non-trivial behavioural property → Rice), **complete bug detection** ("correct on every input" → Rice), and **fully automated verification** ("any program meets any spec" → Rice) are not hard problems; they are *impossible products* — and no amount of cleverness, hardware or AI changes that, because the theorem is constructive. Industry navigates the boundary with the **evidence-based toolkit**: signatures (syntactic → decidable), heuristics and sandboxing (bounded → decidable), testing and fuzzing (evidence on samples), static analysis (restricted decidable models with documented false alarms), code review (human analysis), model checking and bounded verification (finite or bounded → decidable), verified artefacts (one program, one proof), and timeouts/watchdogs (bounded, reportable). Every row of the toolkit is the theory, not an exception to it. The professional's instrument is the **trust ladder**: name the scope, name the evidence, name the residual risk, name the monitoring — and, in reverse, interrogate every vendor claim with the same four questions. Evidence engineering — building the strongest guarantee the theory permits, and stating exactly what it covers — is what the industry has been doing all along, and it is the career skill this unit exists to give you.

4.7 Where you will use this (E.1.4)

- **Security leadership:** antivirus procurement, EDR rollout, "can we guarantee no exfiltration?" — the theorem sets the honest SLA, and the toolkit fills it in.
- **QA and release management:** "bug-free" becomes a scoped, evidenced promise; test strategy, fuzzing campaigns and static analysis gates are justified by the boundary, not by habit.
- **Regulated industries (health, aviation, finance, rail):** certification conversations become scoped-verification conversations — model checking, bounded verification, verified artefacts — with sign-off documents that name scope and residuals.
- **Technical leadership:** when a stakeholder demands a guarantee, you are the person who converts it into a trust-ladder contract instead of an over-promise — the difference between a professional and a liar.
- **Careers:** the toolkit families (security analysis, formal verification, QA engineering, observability) are entire career paths; the theory tells you what each can honestly promise — and the interview question "what can't you automate, and why?" has a crisp, theorem-based answer.

4.8 Self-check (E.1.4)

1. Give the theorem-level reason, in one line each, why perfect virus detection, complete bug detection and fully automated verification are impossible.
2. Name the nine evidence-based toolkit families and, for each, the honest guarantee and the theory that makes it work.
3. Why is signature matching (antivirus) decidable, and why is that not a contradiction with “virus detection is undecidable”? Be precise about which question each answers.
4. Explain why model checking a finite protocol is possible while verifying arbitrary programs is not — connect the answer to Section 2.4’s bounded/unbounded line.
5. Write a trust-ladder contract for “we will make the new payments engine behave identically to the old one” — scope, evidence, residual risk, monitoring.
6. (Challenge) A vendor claims “our AI verifier proves any program correct — it is smarter than Rice’s theorem.” Respond with the constructive argument: what does the theorem hand you for any claimed verifier, and why can no future system escape it?

---

5. Chapter summary and criteria checklist

5.1 The chapter in ten takeaways

1. **Questions come in three kinds:** about *data* (decidable), about a program’s *text* (decidable — linters, scanners, signatures), and about a program’s *behaviour* (undecidable in general) — and the reason is structural: behaviour is an infinite object (every input) and direct inspection is caught in the simulation trap.
2. **The family rule:** any question of the form “does this program behave like so, for all programs or all inputs?” is undecidable; the three named examples — **program equivalence**, **behavioural identity** (the same question under the replacement hat) and **specification satisfaction** — are the professional faces of the family: refactoring, drop-in replacement and verification.
3. **The classification protocol:** (1) state the problem precisely; (2) ask the family question; (3) decidable route — name a total algorithm and *prove it terminates*; (4) undecidable route — reduce from HALT.
4. **The reduction recipe (HALT ≤ X):** suppose an X-solver exists; from any (P, x) build a wrapper P′ whose X-answer is “yes” exactly when P halts on x; then the X-solver decides HALT — contradiction. The two wrapper moves: the **delayed trigger** (property fires at P’s halting) and the **input-forgetter** (P′ ignores its input and runs P on x).
5. **The gallery:** data and text questions are decidable (phone numbers, config equality, `TODO` scans); unbounded behavioural questions are undecidable (equivalence, ERROR-printing, network reads, empty-input halting, totality, virus-ness — each by its own wrapper); bounded behavioural questions are decidable (finish within 2 s — bounded simulation).

6. **Rice's theorem:** any **non-trivial property of a program's behaviour** (of the partial function computed) is undecidable. Trivial means "every program has it or none does"; behaviour means the computed function; the theorem covers exactly those properties, and no others.
7. **The Rice proof is one reduction:** choose a program A with the property; the wrapper W runs P_0 on x_0 and only then behaves like A — so "W has the property" ⟺ "P_0 halts on x_0", and any property-decider would decide HALT (with the two-case fix when the nowhere-defined function itself has the property).
8. **Applying Rice is a three-line routine:** name the property, check behaviour + non-triviality, conclude "undecidable" — instant results for totality, sorting correctness, equivalence to a given program, specification satisfaction, and the rest of the family.
9. **The boundaries of Rice:** syntactic properties (decidable), trivial properties (decidable), and bounded behavioural properties (decidable) escape the theorem; incidental run-time events (printing, file access) are undecidable by *reduction*, not by Rice — saying which theorem handles which question is the E.1.2/E.1.3 exam boundary.
10. **Evidence engineering:** perfect virus detection, complete bug detection and fully automated verification are impossible products; industry ships signatures (syntactic), heuristics and sandboxes (bounded), testing and fuzzing (evidence on samples), static analysis (restricted models with documented false alarms), model checking and bounded verification (finite or bounded), verified artefacts (one program, one proof) and timeouts/watchdogs — every one of them a decidable island, wrapped in the trust ladder of scope, evidence, residual risk and monitoring.

5.2 Criteria checklist (use this to self-test before any assessment)

#	Assessment criterion	You are ready when you can…	Section
E. 1.1	Identify examples of undecidable problems in computing	Name the three kinds of questions and say which is undecidable and why; define program equivalence, behavioural identity and specification satisfaction; give real-world uses and the professional practice for each; classify novel questions into or out of the family using the family rule	1
E. 1.2	Classify a given problem as decidable or undecidable	Run the classification protocol; prove decidability with a total algorithm and a termination argument (and know why "we run it" is not one); prove undecidability with the three-move reduction recipe; build wrappers using the delayed trigger and the input-forgetter on new problems; use the bounded/unbounded line	2
E. 1.3	State and apply Rice's theorem	Define behaviour, behavioural property and non-triviality; state the theorem; sketch the	3

#	Assessment criterion	You are ready when you can…	Section
		proof with the A-wrapper and explain the two-case construction; apply the three-line routine to new properties; name the three boundaries and classify properties into or out of Rice's scope; distinguish Rice from reduction for run-time-event questions	
E. 1.4	Explain the practical implications	Give the theorem-level reason for each impossible product (virus detection, bug detection, verification); name the nine toolkit families with honest guarantees and their theory; explain why signatures are decidable; write a trust-ladder contract; interrogate a vendor's guarantee with the four questions	4

6. Glossary of key terms

Term	Definition
Behaviour (of a program)	The complete input–output relation of a program: the partial function it computes — for each input, the output it produces if it halts; undefined if it runs forever.
Behavioural property	A property of a program determined by the function it computes, not by its text: totality, correctness, "outputs 42 for input 1", equivalence to a given function. Rice's theorem's subject matter.
Behavioural identity	Two programs behave identically if they compute the same function (same output for every input, both halting or both failing to halt on the same inputs); synonym for program equivalence; the drop-in-replacement question.
Bounded behavioural property	A property about a finite prefix of a computation ("halts within 100 steps", "finishes within the budget"); decidable by bounded simulation; outside Rice's scope.
Bounded verification	Proving correctness of a program for all runs up to a given bound (steps, memory); decidable by construction; an industry replacement for universal verification.
Classification protocol	The E.1.2 procedure: state the problem precisely → ask the family question → decidable route (total algorithm + termination proof) or undecidable route (reduction from HALT).
Complete bug detection	The (impossible) product that detects every bug in every program; "correct on every input" is a non-trivial behavioural property, so undecidable by Rice's theorem.

Term	Definition
Decidable problem	A problem with an algorithm that always terminates with the correct answer for every instance.
Delayed trigger	The reduction move that attaches a target property to a program's halting: P′ runs P on x first, and the property fires only when P halts.
Evidence engineering	The professional discipline of producing the strongest guarantee the theory permits — scope, evidence, residual risk, monitoring — for problems whose universal versions are undecidable.
Evidence-based toolkit	The nine industry families that replace impossible guarantees: signatures, heuristics/sandboxing, testing/fuzzing, static analysis, code review, model checking, bounded verification, verified artefacts, timeouts/watchdogs.
Family rule	Any question of the form "does this program (or do these programs) behave like so, for all inputs?" is undecidable in its general, all-programs form.
HALT ≤ X (reduction)	The proof that X is undecidable: suppose an X-solver exists; wrap any (P, x) into an X-instance answering "yes" exactly when P halts on x; the X-solver would then decide HALT — contradiction.
Heuristics	Statistical/behavioural scoring systems (e.g. "looks like malware") that flag suspicious programs with documented scores; evidence-based, not guaranteed.
Input-forgetter	The reduction move for fixed-input variants: P′ ignores its own input and runs P on x, so P′'s behaviour is entirely P-on-x in disguise.
Model checking	Exhaustive verification of a *finite* state space (protocols, finite workflows); decidable because the space is finite; an industry verification tool.
Non-trivial property	A property of programs that some program has and some program does not; the gate for Rice's theorem.
Partial function	A function that may be undefined on some inputs; what a program computes (undefined exactly where the program runs forever).
Perfect virus detection	The (impossible) product that detects all malware, including never-seen malware; "is malicious" is a non-trivial behavioural property, so undecidable by Rice's theorem.
Program equivalence	The property that two programs compute the same function; undecidable by reduction; the refactoring question.
Reduction	The technique that transfers undecidability from a known undecidable problem (HALT) to a new problem X by wrapping instances.

Term	Definition
Rice’s theorem	Every non-trivial property of a program’s behaviour (of the partial function computed) is undecidable.
Sandboxing	Running a program in an isolated, bounded environment to observe its behaviour; decidable because the observation is bounded.
Signature	A syntactic pattern (hash, byte sequence, fingerprint) identifying known malware; signature matching is decidable because it is text matching.
Simulation trap	The failure of “run it and watch” as an analysis method: a program may run forever, and no finite observation distinguishes “slow” from “never”.
Specification satisfaction (VERIFY)	The question of whether a program meets a formal specification; non-trivial behavioural property — undecidable by Rice’s theorem; the verification question.
Static analysis	Analysing a program through a restricted, decidable model (data flow, nulls, taint) — may over-approximate with documented false alarms.
Syntactic property	A property of a program’s text (line count, keywords, structure); decidable by inspection; outside Rice’s scope.
Total program	A program that halts on every input; “is this program total?” is undecidable by Rice.
Trivial property	A property that every program has or no program has; decidable; the gate for Rice’s theorem.
Trust ladder	The honest contract: name the scope, the evidence, the residual risk, and the monitoring.
Undecidable problem	A problem for which no algorithm exists that always terminates with the correct answer; proved by reduction from a known undecidable problem or by Rice’s theorem.
Verified artefacts	Fixed programs with fixed machine-checked proofs (seL4 kernel, CompCert compiler): one program, one proof, finite work.
Wrapper	The program-constructing step of a reduction: the new program P′ built from (P, x) whose behaviour switches on P’s halting.

7. Self-assessment questions (answers in Section 7.6)

7.1 On E.1.1

1. Name the three kinds of questions about inputs, and state which kind is undecidable in general — with the two structural reasons.
2. Define program equivalence precisely, including what must hold for every input.
3. Give one real-world use each for program equivalence, behavioural identity and specification satisfaction, and the professional (non-guaranteed) practice for each.
4. Use the family rule to classify, with a one-line reason: (a) “does this regex match this string?” (b) “do these two programs print the same outputs on all inputs?” (c) “does this program have fewer than 100 lines?” (d) “does this program ever write to the network?”
5. Why can no amount of testing turn equivalence checking into a guaranteed algorithm?

7.2 On E.1.2

1. Run the classification protocol on “does this graph contain a cycle?” and on “does this program ever halt?” — naming the route, the technique and the determination for each.
2. Why is “we run the program and watch” not a termination proof? Name the exact failure.
3. State the reduction recipe in the HALT ≤ X form.
4. Build the full reduction for “does this program ever write to a file named `log.txt`?” — wrapper, equivalence, conclusion.
5. Classify: (a) “does this regex match this text?” (b) “do these two regexes match exactly the same strings?” (c) “does this program halt on the input `0`?” (d) “does this program finish within 5 seconds?” — with technique and one-line reason each.
6. Explain the bounded/unbounded line using “finishes within 2 seconds” versus “finishes eventually”.

7.3 On E.1.3

1. State Rice’s theorem, defining behaviour, behavioural property and non-triviality.
2. Why must a property be non-trivial? Give a trivial property and its decider.
3. Sketch the proof of Rice’s theorem and say where the nowhere-defined function forces the two-case construction.
4. Apply the three-line routine to: (a) “does this program compute a total function?” (b) “does this program compute the identity function?” (c) “does this program satisfy specification S?”
5. Classify each as inside or outside Rice’s scope: (a) “fewer than 50 lines” (b) “halts within 100 steps” (c) “computes the sorting function” (d) “ever prints `ERROR`”.

6. For (d) above, give the correct technique and say why Rice does not apply directly.

7.4 On E.1.4

1. Give the theorem-level reason, one line each, for the impossibility of perfect virus detection, complete bug detection and fully automated verification.
2. Name the nine evidence-based toolkit families, and for three of them give the honest guarantee and the theory behind it.
3. Why is signature matching decidable, and why is that not a contradiction with “virus detection is undecidable”?
4. Why can a finite protocol be model-checked while arbitrary programs cannot be verified?
5. Write a trust-ladder contract for “guarantee the new module behaves identically to the old one”.

7.5 On all criteria (integrated)

1. A fintech QA team must classify: (a) “validate this SA ID number”; (b) “decide whether this payment violates any of the 2 000 fraud rules”; (c) “decide whether the new fraud-rule code will ever loop”; (d) “decide whether two versions of the scoring engine behave identically”. For each: the instance, the technique, the determination, and (where undecidable) the shipped evidence-based solution.
2. A vendor promises “an AI-powered tool that proves any program meets any specification”. Respond with: (a) the theorem that kills it; (b) the constructive counterexample; (c) the four procurement questions; (d) the honest scoped product.
3. Explain why “test suite passed” is evidence, not proof, and name the theorem that makes it so.

7.6 Answers to self-assessment

1. Questions about data (decidable — algorithms process finite data and stop); questions about a program’s text (decidable — text is finite, inspection terminates); questions about a program’s behaviour (undecidable in general — behaviour is an infinite object, one fact per input, and direct inspection is caught in the simulation trap: a run that never ends cannot be distinguished from a slow one).
2. Two programs are equivalent if they compute the same partial function: for every input, either both halt with the same output, or both fail to halt.
3. Equivalence — safe refactoring (“the rewrite behaves like the original”); practice: golden tests, differential testing, review. Behavioural identity — drop-in replacement (“the new supplier’s engine gives the same results”); practice: conformance suites, differential testing. Specification satisfaction — verification and certification; practice: scoped verification (model checking, bounded verification) plus testing — never a universal guarantee.
4. 1. Inside-family question? No — data question; decidable. (b) Behavioural identity for all programs — in the family; undecidable. (c) Text question — out;

decidable. (d) Behavioural — in the family; undecidable (reduction: read after P halts).

5. Because “identical for every input” is a universal claim over infinitely many inputs; any finite test suite covers finitely many of them, so a passed suite cannot logically rule out a difference on an untested input — evidence, not proof.
6. Cycle detection: route = decidable; technique = DFS with a visited set; termination = each edge visited once, then stop; determination = decidable. “Does this program ever halt?”: route = undecidable; technique = it IS HALT; determination = undecidable (Unit D).
7. Running the program is a recogniser: if it halts you know “yes”, but if it has not halted you cannot know whether it ever will — no finite observation decides the “no” answer; a termination proof must be a mathematical argument that the algorithm stops on every instance.
8. Suppose X is decidable (solver S exists). From any (P, x), build a wrapper P′ such that “P′ has property X” ⟺ “P halts on x”. Then S, fed P′, decides HALT — contradiction. X is undecidable.
9. Wrapper: from (P, x) build P′ that runs P on x and, if P halts, writes to `log.txt` and stops. Equivalence: P′ writes `log.txt` ⟺ P halts on x (if P never halts, P′ never writes). A perfect log-writer detector decides HALT — impossible. Undecidable.
10. 1. Decidable — regex/NFA simulation terminates on finite strings. (b) Decidable — convert both to DFAs and check language equality (finite construction, terminates) — a data/text question about two patterns, not behaviour. (c) Undecidable — reduction: P′ ignores its input and runs P on x; “P′ halts on `0`” ⟺ “P halts on x”. (d) Decidable — bounded simulation for exactly the budget; terminates by the bound.
11. “Finishes eventually” is an unbounded behavioural claim: the answer depends on the entire (possibly infinite) computation, so any simulator is a recogniser and the general problem is HALT — undecidable. “Finishes within 2 seconds” is a bounded claim: the first 2 seconds are a finite, completable object; simulate for exactly the budget and the answer is always obtainable — decidable. The wall-clock nuance (machine speed, cache) makes the *guarantee* approximate in practice, but the pure bounded question is decidable.
12. Behaviour = the partial function computed. Behavioural property = a set of such functions. Non-trivial = some program has it, some does not. Rice’s theorem: every non-trivial property of the behaviour of programs is undecidable.
13. Non-triviality is the gate because the trivial properties are decidable trivially: “computes some function” is always true — decided by the program that prints “yes”; its negation always false — decided by the program that prints “no”. If Rice allowed trivial properties it would be false.
14. Choose A with the property; build W: run P_0 on x_0, then behave like A. If P_0 halts, W ≡ A — has the property; if not, W computes the nowhere-defined function ⊥. If ⊥ lacks the property, “W has P” ⟺ “P_0 halts on x_0” — a decider for P decides HALT. If ⊥ has the property, use the non-witness C instead: W′ = run P_0 on x_0 then behave like C; now “W′ has P” ⟺ “P_0 never halts” — still a version of HALT. Contradiction either way.
15. 1. Totality is behavioural (a claim about the function: defined everywhere) and non-trivial (some programs halt everywhere, some do not) — undecidable by

Rice. (b) “Computes the identity function” is behavioural and non-trivial (the identity function is computable; a program that adds 1 does not compute it) — undecidable by Rice. (c) For fixed non-trivial S, “meets S” is behavioural and non-trivial (correct implementations exist; a program that crashes on every input does not meet S — if S is such that even that meets it, choose another non-witness; non-triviality guarantees one) — undecidable by Rice.

16. 1. Outside — syntactic; decidable. (b) Outside — bounded behavioural; decidable by bounded simulation. (c) Inside — behavioural and non-trivial; undecidable. (d) Outside Rice’s direct scope — a run-time event, not a function property — but undecidable by reduction (Section 2.4, Worked 5).
17. Rice’s theorem governs properties of the computed *function*; printing is an event of the *run*, not a property of the function (a function has no side-effects). The correct technique is the reduction: P′ prints `ERROR` after P halts on x, so a perfect ERROR-detector decides HALT. Undecidable — by reduction, not by Rice.
18. Virus detection: “is malicious” is a non-trivial behavioural property — Rice. Complete bug detection: “correct on every input” is a non-trivial behavioural property — Rice. Fully automated verification: “meets S” for fixed S is a non-trivial behavioural property — Rice.
19. Signatures — “detects everything in the signature database, immediately” — syntactic matching is decidable. Heuristics/sandboxing — “flags suspicious behaviour with a documented score” — bounded simulation is decidable. Testing/fuzzing — “no known bug on the tested distribution” — sampling gives evidence, not proof. Static analysis — “catches every instance of the modelled pattern class, with documented false alarms” — restricted decidable models. Code review — “expert inspection” — human analysis. Model checking — “every reachable state of the finite model satisfies the property” — finite space, exhaustive check terminates. Bounded verification — “correct for runs ≤ N” — bounded property, decidable. Verified artefacts — “this specific program provably meets this specific spec” — one program, one proof. Timeouts/watchdogs — “any job over budget is killed and reported” — bounded, reportable.
20. A signature is a syntactic pattern (hash/bytes/behaviour fingerprint of *known* malware); matching text against a pattern is a decidable data/text question. “Is this program a virus?” asks about *behaviour for all programs* — undecidable. The product answers the decidable question (“is this file one of the known ones?”) and never the undecidable one (“is this file malicious in any way?”) — no contradiction; the product simply never claims the second.
21. Model checking explores a finite state space exhaustively — finitely many states and transitions, so the check always terminates: the decidable bounded case. Arbitrary programs have unbounded state (unbounded memory, unbounded runs) — inside the behavioural question, HALT-shaped — so no general verifier can terminate correctly on all inputs. The scope is what makes the tool possible.
22. Scope: “For inputs in the documented production distribution, the new module’s outputs match the old module’s outputs exactly.” Evidence: “Golden test suite of N production-derived cases, differential testing on the distribution, code review of the diff.” Residual risk: “Inputs outside the tested distribution are not covered; resource behaviour (speed, memory) is not part of identity.” Monitoring: “Shadow traffic comparison in staging for 2 weeks; alerts on output divergence in production.”

23. 1. ID string: data — Luhn/format validator, terminates; decidable. (b) (transaction, rules): data — evaluate 2 000 rules, terminates (finite rules, finite fields); decidable. (c) “Will the rule code ever loop?”: behaviour — HALT-shaped; undecidable (reduction); shipped: pattern linters, timeouts, fuzz tests, review. (d) Equivalence of two engines: behaviour — undecidable (reduction with the trivial program); shipped: differential testing, golden files, conformance suites — evidence, not guarantee.
24. 1. Rice’s theorem: “meets specification S” for fixed S is a non-trivial behavioural property. (b) Constructive counterexample: whatever the tool is, take the wrapper W that runs P_0 on x_0 and then behaves like a known-correct program A; the tool must answer about W correctly for every (P_0, x_0) — that is HALT, which cannot be done. (c) Which program classes? Which specs (first-order? temporal?) exactly, and which approximately? What is the false-positive/ negative policy? What is excluded? What bounds apply? (d) The honest product: model checking of finite-state models against safety properties; bounded verification; verified artefacts — with a sign-off that names scope, tool, bound and residuals.
25. A test suite checks finitely many inputs. “Free of bugs” claims correctness on every input — infinitely many — and the “correct on all inputs” property is a non-trivial behavioural property (Rice). A passed suite is therefore evidence about the sampled inputs only; it is valuable, and it is the professional standard, but it is not proof — only scoped verification (finite/bounded classes) proves.

---

8. Exam-style question bank

(Use these to practise under exam conditions; model answers in Section 8.7. Marks are suggestions — your lecturer sets the real weighting.)

8.1 Multiple choice

1. Which kind of question about programs is undecidable in general?
    1. questions about a program’s text (b) questions about a program’s behaviour (c) questions about data (d) questions about file sizes
2. “Do these two programs behave identically on every input?” is:
    1. decidable by running both on a million inputs (b) decidable by comparing their source text (c) undecidable (d) decidable if they are short
3. “Does this program have fewer than 100 lines?” is:
    1. undecidable by Rice’s theorem (b) decidable — a syntactic property (c) undecidable by reduction (d) decidable only for small programs
4. A reduction proving X undecidable must build, from any (P, x), a program P′ such that:
    1. P′ halts on every input (b) P′’s X-answer is “yes” exactly when P halts on x (c) P′ is shorter than P (d) P′ prints `ERROR`

5. “Does this program halt on the empty input?” is:
    1. decidable — just run it (b) undecidable — reduction with the input-forgetter (c) decidable — bounded simulation (d) decidable — the empty input is small
6. “Does this SQL query finish within 5 seconds?” is:
    1. undecidable like HALT (b) decidable — bounded simulation for the budget (c) undecidable by Rice (d) trivial
7. Rice’s theorem applies to:
    1. any question about programs (b) non-trivial properties of a program’s behaviour (c) syntactic properties (d) questions about program length
8. Which property is trivial (and therefore decidable)?
    1. “computes a total function” (b) “computes some partial function” (c) “computes the identity function” (d) “halts within 100 steps”
9. “Does this program compute the sorting function?” is undecidable because:
    1. sorting is hard (b) it is a non-trivial behavioural property — Rice (c) sorting requires comparisons (d) it is a syntactic property
10. Which of the following is decidable?
    1. perfect virus detection (b) complete bug detection (c) signature matching against a malware database (d) verifying that any program meets any specification
11. A perfect virus detector is impossible because:
    1. viruses are too fast (b) “is malicious” is a non-trivial behavioural property (c) malware uses encryption (d) antivirus companies hide their methods
12. Model checking works because:
    1. it simulates programs forever (b) the state space is finite, so the check terminates (c) it uses AI (d) it ignores specifications
13. Which statement is TRUE?
    1. a passed test suite proves a program bug-free (b) a passed test suite is evidence about the tested inputs only (c) testing can be made complete with enough inputs (d) fuzzing proves correctness
14. The wrapper move that builds “P' halts on $\varepsilon \iff$ P halts on x” is called:
    1. the delayed trigger (b) the input-forgetter (c) the diagonal flip (d) the bounded simulation
15. “This program ever prints `ERROR`” is best classified as:
    1. decidable by Rice (b) undecidable by Rice — printing is a function property (c) undecidable by reduction — printing is a run-time event (d) trivial

8.2 Short questions (2–4 marks each)

1. Name the three kinds of questions about inputs and state which is undecidable, with one structural reason (E.1.1).
2. Define program equivalence in one precise sentence, and give one real-world use (E.1.1).
3. State the classification protocol in four steps (E.1.2).
4. State the reduction recipe (HALT $\leq$ X) in two sentences (E.1.2).
5. Give the determination and a one-line reason for: “does this regex match this text?” and “does this program ever write to the network?” (E.1.2).
6. State Rice’s theorem, defining “non-trivial” (E.1.3).

7. Name the three boundaries of Rice's theorem (E.1.3).
8. Give the theorem-level reason why complete bug detection is impossible, and name two toolkit families that replace it (E.1.4).

8.3 Medium questions (5–8 marks each)

1. Build the full reduction for "does this program ever read from stdin?" — wrapper, equivalence, conclusion (E.1.2).
2. Classify with technique and reason: (a) "are these two configuration files semantically identical?" (b) "do these two programs behave identically?" (c) "does this program finish within 100 000 steps on input `0`?" (E.1.2).
3. Explain why "finishes within 2 seconds" is decidable while "finishes eventually" is not, and say where the wall-clock nuance enters (E.1.2).
4. Sketch the proof of Rice's theorem, and explain the role of the nowhere-defined function and the two-case construction (E.1.3).
5. Apply the three-line Rice routine to: (a) "does this program compute a total function?" (b) "does this program compute a function equal to f for a given total f?" (c) "does this program satisfy specification S?" (E.1.3).
6. For each property, classify as decidable/undecidable with technique: (a) "fewer than 100 lines" (b) "halts within 100 steps" (c) "ever prints `ERROR`" — and for (c) say why Rice does not directly apply (E.1.3).
7. Explain why signature matching is decidable while virus detection is not — and what guarantee an antivirus product can honestly make (E.1.4).
8. Why is model checking a finite protocol possible while verifying arbitrary programs is not? Connect your answer to the bounded/unbounded line (E.1.4).
9. Write a trust-ladder contract for "the new payments engine will behave identically to the old one" — scope, evidence, residual risk, monitoring (E.1.4).

8.4 Long questions (10–15 marks each)

1. **The classification triage (E.1.1, E.1.2).** A platform team must classify five requests: (a) "validate this email address"; (b) "check whether two versions of the scoring engine behave identically on every input"; (c) "check whether a SQL query finishes within its budget"; (d) "detect any program that ever exfiltrates data"; (e) "prove the scheduling module meets its formal specification". For each: state the instance precisely, name the technique, give the determination, and for the undecidable ones describe the shipped evidence-based solution with its honest guarantee.
2. **The Rice examination (E.1.3).** (a) State Rice's theorem with all definitions. (b) Prove it for the property "computes a total function" directly (build the wrapper, state the equivalence, conclude). (c) A colleague says "Rice's theorem can't be right — our linter decides 'does this program use `goto`?' perfectly." Respond: which boundary of the theorem protects the linter, and why? (d) Another colleague says "our IDE warns when a function might not return a value — that is behavioural, so it must be impossible." Explain what the IDE actually decides (a decidable, over-

approximated pattern question) and why its occasional false alarms are the theorem, not a bug.

3. **The impossible product (E.1.2, E.1.4).** A startup asks you to build "ShieldAI": a tool that reads any binary and answers, with a guarantee, "is this malware?" (a) Give the reduction showing the perfect version is impossible. (b) Redesign the product honestly: name the decidable components (signatures, sandboxed bounded simulation, pattern heuristics), the evidence methods, and the exact guarantee each provides. (c) Write the two-sentence rejection of the "100% guaranteed" claim, and the one-paragraph honest product description.
4. **The evidence engineer (E.1.4).** A hospital's board demands three guarantees: G1 — every booking request is answered within 2 seconds; G2 — an automated tool guarantees that any new code introduces no infinite loop; G3 — mathematical proof that the scheduling module meets its specification. (a) Classify each guarantee as possible or impossible with a one-line theoretical reason. (b) For G2, redesign the promise honestly (pattern linters, bounded analyses, timeouts — and the SLA's exact wording). (c) For G3, propose the scoped verification strategy (model checking the finite state model, bounded verification of the scheduling horizon, type checking) and the sign-off's contents. (d) Explain to the board in plain language why the scoped guarantee is worth more than the impossible one.

8.5 Applied/scenario question

1. You are the technical lead at an investment bank. The regulator requires "a demonstrated mechanism that guarantees the new portfolio-pricing engine produces the same prices as the current engine for every possible portfolio". The vendor of the new engine claims their product "has been mathematically verified to be behaviourally identical".
    1. Classify the underlying request (E.1.1, E.1.2): what is the exact problem, and what is its determination?
    2. Respond to the vendor's claim: which theorem, which boundary, and which constructive argument show the claim as stated cannot be true (E.1.3)?
    3. Design the honest compliance package: scope the guarantee (which portfolios: the regulator's defined universe), name the evidence (differential testing on the universe's distribution, golden portfolios, property-based tests), the residual risk, and the monitoring (shadow pricing in production, divergence alerts) (E.1.4).
    4. Write the regulator-facing summary in plain language: what is guaranteed, what is evidenced, what is monitored, and why this is the strongest guarantee the theory permits (E.1.4).

8.6 Mark allocation suggestion

Question	Type	Marks	Criteria covered
1–15	MCQ	1 each	E.1.1–E.1.4
16–23	Short	2–4 each	E.1.1–E.1.4

Question	Type	Marks	Criteria covered
24–32	Medium	5–8 each	E.1.2–E.1.4
33–36	Long	10–15 each	E.1.1–E.1.4
37	Applied	15	E.1.1–E.1.4

8.7 Model answers (outline form)

1. 1. — questions about a program's behaviour.
2. 1. — undecidable; equivalence is a universal behavioural claim.
3. 1. — decidable; a syntactic property, outside Rice's scope.
4. 1. — P′'s X-answer is "yes" exactly when P halts on x.
5. 1. — undecidable; the input-forgetter wrapper (P′ runs P on x, ignoring its own input).
6. 1. — decidable; bounded simulation for the budget terminates.
7. 1. — non-trivial properties of a program's behaviour.
8. 1. — "computes some partial function" holds of every program; decidable trivially.
9. 1. — a non-trivial behavioural property, by Rice's theorem.
10. 1. — signature matching is syntactic and decidable.
11. 1. — "is malicious" is a non-trivial behavioural property.
12. 1. — finite state space, exhaustive check terminates.
13. 1. — a passed suite is evidence about the tested inputs only.
14. 1. — the input-forgetter.
15. 1. — undecidable by reduction; printing is a run-time event, not a function property.
16. Data (decidable — finite processing), text (decidable — finite inspection), behaviour (undecidable — infinite behaviour over all inputs; simulation trap).
17. Two programs are equivalent if they compute the same partial function (same outputs on the same halting inputs). Use: safe refactoring — "the rewrite behaves like the original".
18. 1. State the problem precisely; (2) ask the family question (data/text → decidable route; behaviour → undecidable route); (3) decidable: find a total algorithm and prove termination; (4) undecidable: reduce from HALT.
19. Suppose X is decidable. From any (P, x) build a wrapper P′ whose X-answer is "yes" exactly when P halts on x; then an X-solver decides HALT — contradiction, so X is undecidable.
20. Regex-vs-text: decidable — NFA simulation terminates on finite strings. "Ever writes to the network": undecidable — reduction (write after P halts); a perfect detector would decide HALT.
21. Every non-trivial property of a program's behaviour (of the partial function computed) is undecidable; non-trivial means some program has it and some does not.
22. Syntactic properties (decidable); trivial properties (decidable); bounded behavioural properties (decidable).

23. “Correct on every input” is a non-trivial behavioural property — Rice. Replacements: testing/fuzzing (evidence), static analysis (restricted decidable models), code review (human analysis), bounded verification.
24. Wrapper: P′ runs P on x, then performs a read from stdin. “P′ reads from stdin” ⟺ “P halts on x” (P′ reads only after P halts). A perfect stdin-read detector decides HALT — impossible. Undecidable.
25. 1. Decidable — data question; parse into sets and compare, terminates. (b) Undecidable — behavioural; reduction with the trivial program Z (P′ behaves like Z iff P halts on x). (c) Decidable — bounded behavioural property; simulate 100 000 steps, terminates by the bound.
26. “Finishes eventually” is unbounded: the answer depends on the whole (possibly infinite) run, so simulation is a recogniser — the problem is HALT, undecidable. “Finishes within 2 seconds” is bounded: the first 2 seconds are finite and completable; simulating exactly the budget always terminates with the answer. Nuance: as *wall-clock* time the guarantee depends on a performance model (machine speed, cache) — engineers package the decidable step-budget question with query limits and `EXPLAIN`.
27. Choose A with property P. W: run P_0 on x_0; if it halts, behave like A. P_0 halts $\Longrightarrow$ W ≡ A $\Longrightarrow$ W has P. P_0 never halts $\Longrightarrow$ W ≡ $\bot$ (nowhere-defined). If $\bot$ lacks P, “W has P” ⟺ “P_0 halts on x_0” — a P-decider decides HALT. If $\bot$ has P, use a non-witness C instead: W′ = run P_0 on x_0, then behave like C; “W′ has P” ⟺ “P_0 never halts” — still HALT. Contradiction either way; no decider for P exists.
28. 1. Totality: behavioural, non-trivial — undecidable. (b) “Equals f”: behavioural (equality of functions), non-trivial (some programs compute f, others do not) — undecidable. (c) “Meets S”: behavioural, non-trivial for non-trivial S — undecidable.
29. 1. Decidable — syntactic. (b) Decidable — bounded simulation. (c) Undecidable — by reduction, not Rice: printing is a run-time event, not a function property; P′ prints after P halts.
30. Signature matching is text matching against known patterns — a decidable syntactic question (Boundary 1). Virus detection asks about behaviour for all programs — non-trivial behavioural — undecidable. Honest guarantee: “detects everything in our signature database immediately, sandbox-scores the rest with documented thresholds”; never “detects all malware”.
31. Model checking exhausts a finite state space — finitely many states, so the check always terminates (decidable, bounded). Arbitrary programs have unbounded state and unbounded runs — the behavioural question, HALT inside — so no general verifier can terminate correctly on all inputs. Scope (finiteness) is what makes the tool possible.
32. Scope: “For inputs in the documented production distribution, outputs match the old engine exactly.” Evidence: “Golden portfolio suite; differential testing on the distribution; property-based tests; review of the diff.” Residual risk: “Inputs outside the distribution; resource behaviour; timing.” Monitoring: “Shadow pricing in staging and production; divergence alerts; rollback plan.”
33. 1. Email: data — regex validator, terminates — decidable. (b) Equivalence of engines: behaviour, all inputs — undecidable — ship differential testing + golden files (evidence, scoped to the distribution). (c) Query within budget:

bounded — decidable — bounded simulation/query limits. (d) “Ever exfiltrates”: behaviour — undecidable (reduction) — ship network policy + sandboxing + monitoring (constraints on the environment, decidable). (e) “Meets spec”: non-trivial behavioural — undecidable in general — ship scoped verification: model checking the finite state model, bounded verification of the horizon, plus testing; sign-off names scope and residuals.

34. 1. Statement with definitions (behaviour = partial function; behavioural property = set of functions; non-trivial = some program has it, some does not; conclusion: every non-trivial behavioural property is undecidable). (b) A = any total program. W: run P_0 on x_0, then behave like A. P_0 halts $\Longrightarrow$ W total; P_0 never halts $\Longrightarrow$ W never halts on any input (not total) — “W total” $\Longleftrightarrow$ “P_0 halts on x_0”; a totality-decider decides HALT. (c) The linter checks a *syntactic* property (text contains `goto`), which is outside Rice’s scope — Boundary 1: decidable by inspection. (d) The IDE checks a *restricted decidable model* (data-flow over the function’s text) and answers “will every path return?” approximately — it over-approximates (reports some false alarms) so it never misses; “occasional false alarm” is the price of the decidable approximation, exactly as the theorem predicts for any behavioural-looking tool.

35. 1. Reduction: suppose a perfect malware detector M exists. From any (P, x) build P' that runs P on x and, if P halts, performs a clearly malicious act (deletes a file). “P' is malware” $\Longleftrightarrow$ “P halts on x” — M decides HALT. Impossible. (b) Decidable components: signature matching (syntactic — decidable), sandboxed bounded simulation (bounded — decidable), pattern heuristics with documented scores (evidence); each with its exact guarantee: “detects everything in the database”, “flags behaviours observed in a bounded sandbox run”, “scores suspicious patterns per the published model”. (c) Rejection: “Any tool that guarantees detection of all malware would decide the Halting Problem, which is impossible — your claim is marketing.” Honest product: “ShieldAI detects all signatures in its database instantly, sandboxes and scores unknown binaries against published behavioural models, and is monitored 24/7 with incident response — no product can promise more.”

36. 1. G1: possible — decidable validation pipeline with a time budget (engineering, not theory). G2: impossible — perfect loop detection is HALT; no tool can guarantee it for arbitrary code. G3: possible only scoped — “meets spec” for arbitrary programs is Rice-undecidable; scoped verification is possible. (b) Honest G2: CI gates with pattern linters (while-true without exit, unbounded recursion patterns), bounded analyses, per-test timeouts; SLA: “flags known loop patterns in the supported language subset; does not claim completeness; every CI job runs under a timeout.” (c) G3: model check the finite booking state machine for the safety invariants (decidable — finite); bounded verification of the scheduling horizon; type checking; sign-off names the model, the tool, the bound, the residuals. (d) Plain language: the impossible guarantee is a promise that cannot be kept; the scoped guarantee is kept every time — the industry ships safety-critical systems exactly this way, and regulators rely on scope + evidence + monitoring, which is the strongest honest contract.

37. 1. The request is program equivalence (“the new engine behaves identically to the current engine on every possible portfolio”) — a universal behavioural

claim; undecidable by reduction (equivalence to a given program is also Rice-undecidable). (b) “Mathematically verified to be behaviourally identical” as a universal claim cannot be true: equivalence is a non-trivial behavioural property (Rice); constructively, wrap any (P, x) in an engine that prices like the current one only if P halts on x — a perfect equivalence verifier would decide HALT. (c) Compliance package: Scope — “identical prices for portfolios in the regulator’s defined universe (documented distribution and constraints)”. Evidence — differential testing across the universe’s distribution; golden portfolio suite; property-based tests; review of the pricing model diff. Residual risk — portfolios outside the defined universe; numeric-precision edge cases within tolerance. Monitoring — shadow pricing in production; divergence alerts; quarterly re-validation. (d) Plain-language summary: “We guarantee identical prices across the regulated universe, evidenced by differential testing on that universe, with production shadow pricing monitoring any divergence — the strongest guarantee the theory permits; a stronger claim would be impossible for any product, including the vendor’s.”

---

9. Sources and further reading

Standard textbooks

- Sipser, M. (2013). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning. — Chapter 5 (reducibility: the Halting Problem and its relatives, mapping reductions) and Chapter 6 (Rice’s theorem and its proof; the recursion theorem). The definitive treatment of everything in this unit.
- Hopcroft, J. E., Motwani, R. & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Addison-Wesley. — Chapter 9 (undecidability, reduction, and Rice’s theorem as an application of the recursion-theoretic tools).
- Papadimitriou, C. H. (1994). *Computational Complexity.* Addison-Wesley. — Chapter 3 (undecidability and Rice’s theorem in the context of the computability/complexity boundary).

Foundational and historical context

- Rice, H. G. (1953). *Classes of Recursively Enumerable Sets and Their Decision Problems.* Transactions of the American Mathematical Society, 74(2), 358–366. — The original paper stating and proving the theorem that bears Rice’s name.
- Turing, A. M. (1936). *On Computable Numbers, with an Application to the Entscheidungsproblem.* Proceedings of the London Mathematical Society, s2-42(1), 230–265. — The source of HALT, the problem every reduction in this unit leans on.
- Davis, M. (ed.) (1965). *The Undecidable: Basic Papers on Undecidable Propositions, Unsolvable Problems and Computable Functions.* Dover. — The foundational papers (Gödel, Church, Turing, Post, Rice) in one volume.

Reference pages consulted for fact-checking (2026)

- Wikipedia: “Rice’s theorem”, “Halting problem”, “Reduction (recursion theory)”, “Undecidable problem”, “Program equivalence”, “Formal verification”, “Model checking”, “Bounded verification”, “Antivirus software” (detection methods: signatures, heuristics, sandboxing), “seL4”, “CompCert”, “Differential testing”, “Fuzzing”, “Static program analysis”. *(Used to verify the theorem statements, the standard reduction constructions (equivalence, empty-input halting, totality), the boundaries of Rice’s theorem (syntactic, trivial, bounded properties), and the industry practices named in the evidence-based toolkit. Cross-checked against Sipser where possible.)*

Module documents (source of truth for outcomes)

- TCR117V_Revised_Learning_Outcomes.md — Outcome E.1 and criteria E.1.1–E.1.4.
- Study guide 2026 — module purpose (Section 6) and outline (Section 23, topic E).
- Unit B study notes — Church’s thesis and the diagonal argument (Sections 5–7), the foundation this unit builds on.
- Unit D study notes — the Turing machine, recognisers and deciders, the Halting Problem, diagonalisation, and the reduction recipe (Sections 4–8), which this unit reuses as its working tools.

End of Unit E study notes. Next unit: F — Finite automata and regular expressions (combined).